

CORSO DI BIG DATA — TOPIC 7: IOT STREAM ANALYTICS

Self Healing Fleet

Ingestion continua, detection real-time e analisi predittiva offline
su una flotta di AGV in un magazzino

Autore: Marcello Tascioni

Data: 21 luglio 2026

Repository: self-healing-fleet

Indice

1. Obiettivi del progetto

- 1.1 Contesto e vincoli del corso
- 1.2 Il problema iniziale: la mappa del magazzino come grafo
- 1.3 Obiettivi funzionali
- 1.4 Un progetto di Big Data, non di robotica

2. Architettura e tecnologie utilizzate

- 2.1 Vista d'insieme: architettura lambda
- 2.2 Sorgente dati: ROS1 Noetic + Gazebo + TurtleBot3
- 2.3 Contratto dati: schema del messaggio di telemetria
- 2.4 Ingestion: Apache Kafka
- 2.5 Due modalità di simulazione: flotta reale e generatore sintetico
- 2.6 Speed layer: detection real-time in Spark Structured Streaming
- 2.7 Batch layer: persistenza e soglie adattive
- 2.8 Analisi predittiva offline (time series)
- 2.9 Serving layer: Spark SQL e sistema TAG con Hugging Face
- 2.10 Presentazione: backend Node.js e dashboard
- 2.11 Estensione: self healing sulla flotta reale (e previsione live, Passo 15/16)
- 2.12 Struttura del database (storico Parquet)
- 2.13 Il percorso dei dati: dalla sorgente alla visualizzazione
- 2.14 Orchestrazione e deployment
- 2.15 Pulizia e validazione dei dati lungo la pipeline
- 2.16 La suite di verifica (test/): cosa controlla, come

3. Risultati

- 3.1 Effectiveness: precisione della detection, della previsione e del TAG
- 3.2 Efficiency: scalabilità e latenza
- 3.3 Valutazione sulla scalabilità
- 3.4 Verifica end-to-end sulla flotta reale
- 3.5 Copertura dei requisiti del corso

4. Osservazioni conclusive

- 4.1 Lezioni tecniche
- 4.2 Limiti dichiarati
- 4.3 Sviluppi futuri

1. Obiettivi del progetto

1.1 Contesto e vincoli del corso

Il progetto è stato sviluppato per il corso di Big Data, in relazione al **Topic 7 — IoT Stream Analytics**. Il tema scelto è la gestione di una flotta di AGV (Automated Guided Vehicle) in un magazzino: i veicoli percorrono una mappa a grafo trasportando merci, inviando in continuo la propria telemetria (posizione, velocità, stato dei task, canali di salute del motore/batteria). Il sistema deve ingerire questo flusso continuo, rilevarne le anomalie in tempo reale, prevederne i guasti su base storica, e rispondere a interrogazioni in linguaggio naturale.

Il vincolo di fondo, esplicitato fin dall'inizio nella documentazione di progetto, è che si tratta di un **progetto di Big Data, non di robotica**: il robot è unicamente la sorgente del flusso dati. Le tre "V" classiche sono tutte presenti nel disegno del sistema — *Volume* (il generatore sintetico permette di scalare da poche unità a migliaia di sorgenti simulate), *Velocity* (ingestion e detection in streaming, non batch), *Veracity* (dati fisicamente realistici, con rumore e vincoli reali imposti da un simulatore fisico, non generati "a tavolino") — a cui si aggiunge *Value*, dato dal layer di analisi predittiva che trasforma lo storico grezzo in previsioni azionabili.

1.2 Il problema iniziale: la mappa del magazzino come grafo

Prima ancora di parlare di dati o di pipeline, il progetto richiede di rispondere a una domanda di modellazione: **come si rappresenta un magazzino per una flotta di AGV?** La risposta standard nella logistica robotica — adottata anche qui — è un **grafo roadmap**: non una mappa metrica continua (che richiederebbe SLAM, irrilevante per gli scopi del progetto, §1.4), ma un insieme discreto di *nodi* (punti significativi: incroci, banchine di carico, scaffalature, stazioni di ricarica) collegati da *archi* (i percorsi realmente disponibili fra loro). Un robot non "naviga liberamente": sceglie una sequenza di nodi da raggiungere ed è il pianificatore locale (§2.2) a occuparsi di come muoversi lungo ciascun arco.

Questa scelta di modellazione ha una conseguenza diretta sul problema di Big Data: se ogni arco può ospitare al più un certo numero di robot contemporaneamente (il suo `capacity`), un arco con `capacity=1` è un **corridoio a corsia singola** — un vincolo fisico reale che due robot possono violare solo restando bloccati in attesa reciproca (deadlock) o cedendosi il passo all'infinito senza avanzare (livelock). Questi due corridoi a capacità singola non sono un dettaglio implementativo:

sono il meccanismo scelto *di proposito* per generare, in modo controllato e riproducibile, gli scenari comportamentali che il sistema deve rilevare (§2.6, tabella di verità in §3.1).

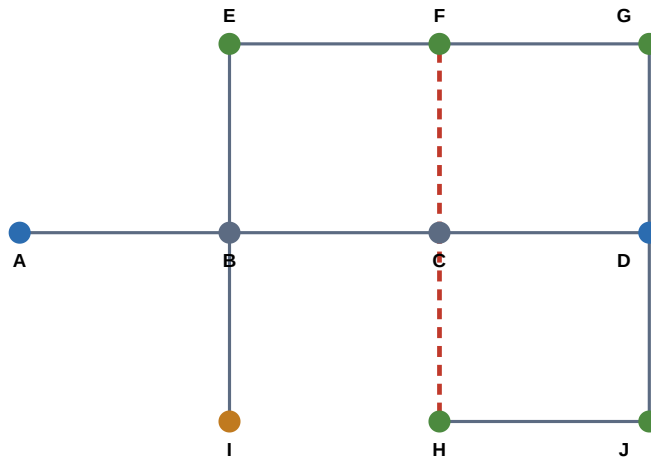


Fig. 0 — Il grafo del magazzino (`config/warehouse_graph.json`): 10 nodi, 11 archi. Legenda nodi — ● dock (A, D), ● incrocio (B, C), ● storage (E, F, G, H, J), ● ricarica (I). Archi grigi = corsia doppia (`capacity=2`); archi rossi tratteggiati (C-F, C-H) = **corsia singola** (`capacity=1`), i due choke point che rendono possibili deadlock/livelock sull'anello di storage F-G-J-H.

Nel messaggio di telemetria (schema completo in §2.3) questo grafo si riflette in due campi: `current_edge` (su quale arco si trova il robot in questo istante, calcolato proiettando la posizione fisica (x, y) sull'arco più vicino) e `goal_node` (il prossimo nodo della sequenza assegnata). È lo stesso identico grafo — stesso file, stesse coordinate — a essere: la roadmap che i robot seguono in Gazebo (§2.2), i vincoli di traffico che il job di detection sorveglia (§2.6), e ciò che la dashboard disegna a video (§2.10, con lo stesso layout della Fig. 0). Un'unica fonte di verità, non tre rappresentazioni da tenere sincronizzate a mano.

1.3 Obiettivi funzionali

1. **Ingestion continua e realistica.** La sorgente dati doveva essere una vera pipeline ROS + Gazebo + TurtleBot3, non un generatore puramente sintetico fin dal principio — requisito esplicito del progetto per garantire dati con vincoli fisici reali (odometria, lidar, dinamica del robot), non solo formalmente conformi allo schema.
2. **Detection in tempo reale** di due categorie di anomalie: anomalie di *salute* (derive termiche, picchi di corrente, collasso della batteria, sensori bloccati) e anomalie *comportamentali* (deadlock e livelock su corridoi a corsia singola).
3. **Analisi predittiva offline** sullo storico dei canali di salute, per stimare *quando* un robot supererà una soglia critica (tempo di anticipo, o *lead time*), non solo se è già in anomalia.
4. **Riduzione dei falsi positivi** tramite un meccanismo di soglie adattive, calibrate sullo storico reale usando i guasti iniettati come ground truth.
5. **Interrogazione in linguaggio naturale** dello storico accumulato, tramite un layer text-to-SQL (TAG, Table-Augmented Generation) basato su un LLM open-source.
6. **Quattro tecnologie obbligatorie**, tutte presenti almeno in versione minima: Apache Kafka, Spark Structured Streaming, un modello di previsione su serie storiche, un LLM.
7. **Valutazione sperimentale** quantitativa, sia di *effectiveness* (precisione della detection e delle previsioni, accuratezza del TAG) sia di *efficiency* (scalabilità del throughput, latenza end-to-end), con un meccanismo di ground truth ('injected_faults') che rende le metriche calcolabili in modo oggettivo, non stimato.

1.4 Un progetto di Big Data, non di robotica

Questo vincolo ha guidato diverse scelte architetturali documentate nel seguito: la navigazione dei robot non usa SLAM/mappa statica (irrelevante per gli obiettivi del progetto, il grafo del magazzino è già una roadmap nota); i canali di salute (temperatura motore, corrente, percentuale batteria) non esistono nel TurtleBot3 simulato e vengono *sintetizzati* nel nodo-ponte ROS → Kafka a partire da parametri nominali configurabili; l'iniezione dei guasti avviene sommando una firma parametrica alla telemetria realistica, non generando dati fittizi da zero. In un'estensione realizzata oltre il piano originale (§2.11), è stata anche introdotta un'unica, dichiarata eccezione a questo principio: per la sola flotta reale, un'anomalia di salute rilevata chiude l'anello con un'azione minima (invio verso un nodo di riparazione) — la decisione applicativa resta comunque nel backend, non nel robot.

2. Architettura e tecnologie utilizzate

2.1 Vista d'insieme: architettura lambda

Il sistema è organizzato come una **architettura lambda**: gli stessi dati grezzi (il topic Kafka `telemetry`) alimentano sia un percorso *speed* a bassa latenza per la detection in tempo reale, sia un percorso *batch* più lento ma completo per l'analisi storica e predittiva. Un *serving layer* espone entrambi i risultati, sia alla dashboard live sia alle interrogazioni ad-hoc in linguaggio naturale.

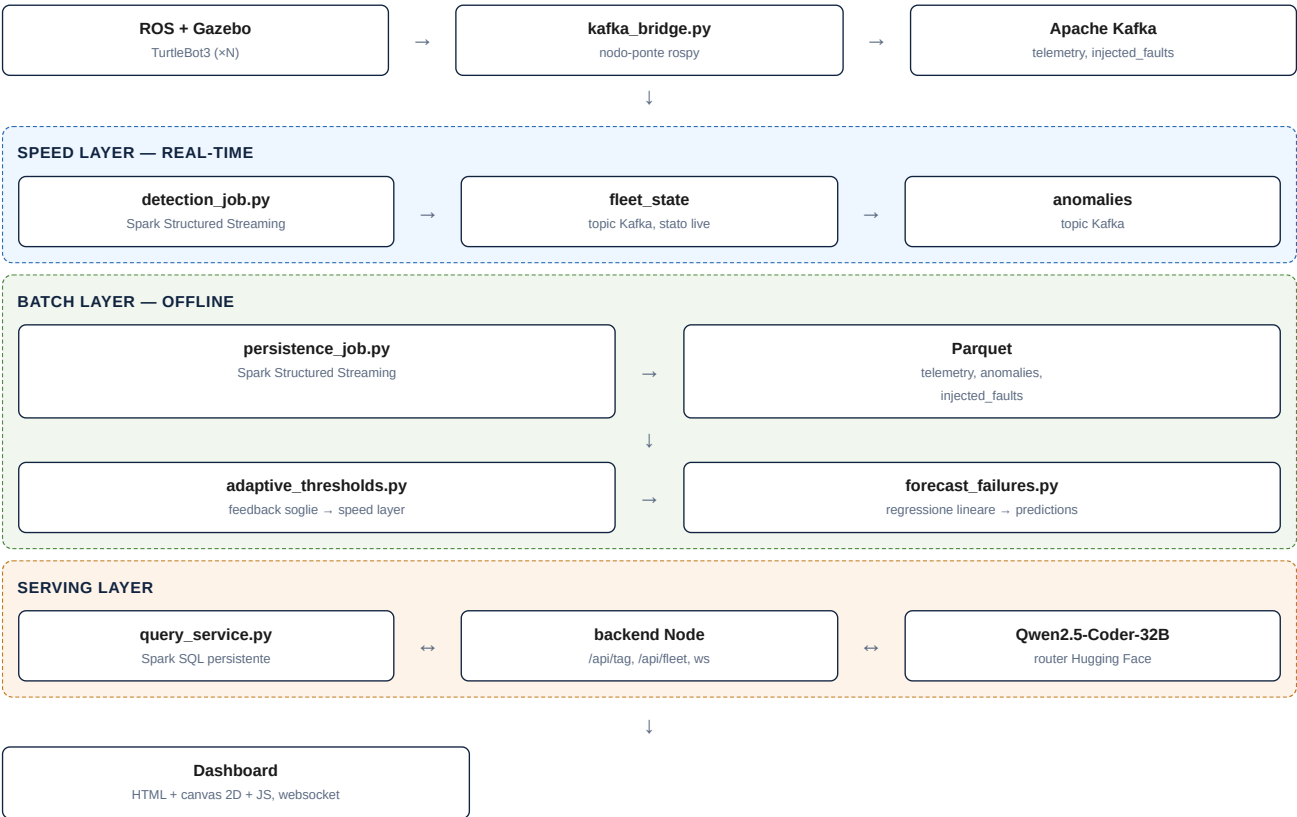


Fig. 1 — Architettura lambda del sistema: gli stessi dati grezzi alimentano il percorso real-time (speed layer) e quello storico (batch layer); il serving layer li espone entrambi alla dashboard e alle interrogazioni in linguaggio naturale.

Perché lambda e non kappa

La distinzione non è terminologica: cambia dove vive la logica e quante volte viene scritta. Un'architettura **kappa** avrebbe un solo motore di elaborazione (lo streaming) e tratterebbe lo storico come un semplice *replay* del log — nessuna logica duplicata, nessuna vista costruita due volte con tecniche diverse. Qui non è così, per una ragione concreta: il segnale di **rischio** (tabella sopra) viene calcolato *due volte*, con due tecniche diverse, in due punti diversi della pipeline:

- nella **speed layer** (§2.6), il quarto operatore stateful di `detection_job.py` emette un preavviso appena un canale supera una soglia morbida abbastanza spesso in una finestra di 60s — una regola euristica, a bassa latenza, con un `lead_time_s` stimato in modo grossolano (la finestra stessa, non un'estrapolazione);
- nella **batch layer** (§2.8), `predictive/forecast_failures.py` allena una vera regressione lineare sullo storico Parquet e calcola analiticamente l'istante di superamento soglia — più lento (gira on-demand, non ad ogni messaggio) ma più accurato.

Sono due **viste dello stesso concetto**, con un compromesso latenza/accuratezza opposto — esattamente la dualità speed-view/batch-view che definisce lambda, non un dettaglio implementativo incidentale. Il serving layer (qui: il pannello "Robot a rischio" della dashboard, §2.10) le **unisce a runtime**, dando precedenza alla vista live quando presente perché più fresca, coerentemente con l'idea lambda che la vista batch è quella "definitiva" ma la vista speed copre la finestra di latenza fino a quando la prima non è ancora arrivata.

Detto questo, il sistema non è un'istanza da manuale enciclopedico di lambda: la persistenza storica (`persistence_job.py`, §2.7) usa la *stessa* tecnologia della speed layer (Spark Structured Streaming, non un motore batch separato come Hadoop MapReduce nell'architettura lambda originale di Marz), e legge da Kafka con `startingOffsets=earliest` — cioè tratta il log Kafka come sorgente di verità ri-processabile, un'idea più tipica di kappa. È quindi più corretto descriverlo come **un'architettura lambda con una persistenza in stile kappa**: due viste concettualmente distinte (rischio real-time vs rischio batch) sopra un'unica tecnologia di elaborazione (Spark Structured Streaming) e un'unica sorgente ri-leggibile (Kafka) — un ibrido pragmatico più che l'uno o l'altro pattern in forma pura.

2.2 Sorgente dati: ROS1 Noetic + Gazebo + TurtleBot3

La sorgente dati è una simulazione fisica, non un generatore di numeri casuali conforme a uno schema. Ogni robot è un TurtleBot3 "burger" simulato in Gazebo, controllato tramite lo stack di navigazione ROS standard (`move_base` con pianificatore locale DWA). La navigazione **non usa SLAM né una mappa statica/AMCL**: il costmap globale vive nel frame `odom` (60×40 m, nessun `static_layer`) — scelta deliberata, perché il grafo del magazzino (§2.3) è già una roadmap nota a priori, e introdurre SLAM avrebbe aggiunto complessità robotica irrilevante per gli obiettivi di Big Data del progetto. Ogni robot riceve in sequenza i nodi del proprio percorso come goal `actionlib`.

Multi-robot e scenari di conflitto

TurtleBot3 non supporta nativamente il multi-robot: è stato necessario uno script (`gen_robot_description.sh`) che inietta il namespace nei plugin Gazebo e rinomina esplicitamente i frame TF (a differenza dei topic, `tf2` non prefissa automaticamente i `frame_id` in base al namespace di lancio). La flotta di riferimento comprende 3 robot attivi (R1-R3) più un quarto robot di riserva (R4, §2.11); uno scenario esteso ("large") raddoppia la flotta attiva a 6 robot con due riserve (R4, R8), selezionabile con un parametro `scale` del file di lancio ROS.

Sintesi dei canali di salute e fault injection

Il nodo-ponte `kafka_bridge.py` sottoscrive `/odom`, `/cmd_vel`, `/scan`, `/move_base/goal`, `/move_base/result`. I tre canali di salute (`battery_pct`, `motor_current`, `motor_temp`) — assenti dal TurtleBot3 simulato — vengono sintetizzati a partire da parametri nominali condivisi (`health_channels_nominal` in `config/experiment.json`): la batteria si scarica in funzione dello stato di moto/idle e si ricarica in "charging", corrente e temperatura oscillano attorno a un valore nominale con rumore gaussiano. La posizione sull'arco corrente del grafo (`current_edge`) è calcolata proiettando (`x`, `y`) reali sul grafo.

L'iniezione dei guasti è realizzata da una classe `FaultInjector` che legge un `fault_schedule` (§2.3) e **somma una firma parametrica alla telemetria realistica prima della pubblicazione**, non genera un canale separato: un guasto iniettato attraversa esattamente lo stesso percorso dati di un guasto "vero". Le firme implementate sono cinque, tutte parametriche (la quinta aggiunta al Passo 15, §2.11, per la previsione live):

Firma	Canale	Meccanismo
<code>deriva_termica</code>	<code>motor_temp</code>	rampa lineare fino a un plateau
<code>spike_corrente</code>	<code>motor_current</code>	picco che resta elevato per la durata del guasto
<code>batteria_collasso</code>	<code>battery_pct</code>	discesa più ripida del drain rate nominale
<code>sensore_bloccato</code>	configurabile	valore congelato all'ultimo campione valido
<code>preavviso_intermittente</code>	configurabile	raffiche saltuarie oltre una soglia "morbida", nominale in mezzo — non un guasto continuo

Un sesto meccanismo, `rumore_sensore` (Passo 16, §2.11), aggiunge rumore gaussiano extra su un canale con la stessa via reattiva — ma **non** è una firma di guasto: deliberatamente esclusa da `injected_faults`, serve a generare falsi positivi controllati per il job di calibrazione delle soglie adattive (§2.7), non a far parte del ground truth di guasti valutato in §3.2.

A queste si affiancano due **scenari comportamentali** (deadlock, livelock), realizzati non come firme sui dati ma come vera e propria contesa di traffico: il grafo del magazzino include due archi a corsia singola (`capacity=1`, C-F e C-H). In un'estensione realizzata dopo il completamento del piano originale (§2.11), questi due archi sono stati fisicamente ristretti anche in Gazebo (pareti statiche SDF, corridoio libero di 0.8 m), così che i conflitti di traffico siano osservabili come vincoli fisici reali e non solo come un numero nel file di configurazione.

La routine di un robot: dallo spawn alla missione

Ogni robot con un compito assegnato (`tasks[]` in `config/experiment.json`) nasce fermo sul proprio `start_node`; se il task specifica un `start_time_s` diverso da zero, resta inerte fino a quell'istante (utile per far scattare missioni e guasti in sequenze temporali controllate durante gli esperimenti). Allo scadere, `graph_navigator.py` — un nodo rospy dedicato per robot, separato da `kafka_bridge.py` — inizia a eseguire la sua `goal_sequence` un nodo del grafo alla volta: per ciascuno calcola la posa target dalle coordinate del grafo, la invia come `MoveBaseGoal` a `move_base` tramite un `actionlib.SimpleActionClient`, e attende il risultato con un timeout proporzionale alla distanza (non un timeout fisso: un arco lungo ha naturalmente bisogno di più tempo). Raggiunto un nodo (o mancato: loggato, non bloccante) passa al successivo, fino a fine sequenza. Un robot senza task (una riserva, es. R4) nasce e resta fermo, in attesa passiva di un comando: non è "spento", semplicemente non ha nulla da fare finché non gli viene assegnata una missione dal vivo.

`graph_navigator.py` e `kafka_bridge.py` non si parlano direttamente fra loro: il primo conosce solo la sequenza di goal e lo stato di `move_base`; il secondo ricostruisce `task_state` (idle/moving/blocked/charging) osservando passivamente gli stessi topic ROS standard che `move_base` pubblica comunque (`/move_base/goal`, `/move_base/result`, `/odom`), non un canale dedicato inventato per l'occasione.

Due canali di comando indipendenti possono intervenire in qualunque momento, senza che i due nodi si accorgano l'uno dell'altro:

Topic	Nodo in ascolto	Payload	Effetto
<code>~nav_control</code>	<code>graph_navigator.py</code>	<code>{"nodes": [...]}</code>	annulla il goal corrente e sostituisce la missione con la nuova sequenza di nodi
<code>~nav_control</code>	<code>graph_navigator.py</code>	<code>{"cmd": "freeze"}</code>	annulla il goal corrente e non assegna nulla: il robot resta fermo dov'è
<code>~fault_inject</code>	<code>kafka_bridge.py</code>	<code>{"fault_type", "duration_s", "params?"}</code>	aggiunge un guasto allo schedule interno, applicato dal prossimo tick di telemetria

Entrambi i topic sono pubblicati da `fleet_control_service.py` (§2.11) su un nome assoluto namespaced per robot (es. `/R1/graph_navigator/nav_control`), lo stesso servizio HTTP che la dashboard chiama per avviare/fermare la simulazione, iniettare un guasto o riassegnare una missione: il robot non distingue se un comando arriva dalla logica automatica di self healing o da un click dell'operatore, il payload è identico in entrambi i casi.

Dai messaggi ROS al messaggio Kafka: la trasformazione passo per passo

`kafka_bridge.py` non inventa nulla: ogni campo del messaggio di telemetria nasce da un messaggio ROS standard già pubblicato da Gazebo/ `move_base` , o da una sintesi dichiarata esplicitamente come tale. La provenienza esatta:

Topic ROS	Tipo messaggio	Campi letti	Diventano nella telemetria
/odom	nav_msgs/Odometry	pose.pose.position (x,y), pose.pose.orientation (quaternione), twist.twist.linear.x , twist.twist.angular.z	x , y , theta (quaternione → yaw via tf.transformations.euler_from_quaternion), v_lin , v_ang
/cmd_vel	geometry_msgs/Twist	linear.x , angular.z	cmd_v_lin , cmd_v_ang — velocità comandata, non quella reale: utile a valle per confrontare comando e risposta fisica
/scan	sensor_msgs/LaserScan	ranges[] , filtrato per range_min / range_max	min_obstacle_dist — il minimo dei valori validi
/move_base/goal	move_base_msgs/MoveBaseActionGoal	goal.target_pose.pose.position (x, y)	goal_node — nodo del grafo più vicino alla posa target (nearest_node)
/move_base/result	move_base_msgs/MoveBaseActionResult	solo l'arrivo del messaggio, non il contenuto	azzerà have_active_goal , un ingrediente di task_state

Due campi non provengono da nessun singolo messaggio ROS, ma da una trasformazione esplicita sui dati sopra:

- current_edge** : (x, y) non cade mai esattamente su un arco del grafo (fisica reale, evitamento ostacoli), quindi `nearest_edge()` proietta il punto su ciascun segmento nodo-nodo (clampato fra i due estremi) e sceglie l'arco a distanza minima — non il nodo più vicino, l'arco, perché è la capacità dell'arco (corridoio a corsia singola) che conta per deadlock/livelock, non la vicinanza a un nodo.
- task_state** : una piccola macchina a stati che combina tre segnali nel tempo, non un singolo topic — `idle` (nessun goal attivo, lontano da un nodo di ricarica), `charging` (nessun goal attivo, entro `CHARGING_RADIUS_M=0.5m` da un nodo `kind="charging"`), `moving` (goal attivo, velocità sopra `VEL_EPS=0.02 m/s` in almeno una componente negli ultimi `BLOCKED_AFTER_S=5s`), `blocked` (goal attivo ma fermo da più di 5s). I tre parametri non sono arbitrari: `VEL_EPS` filtra il rumore numerico dell'odometria attorno allo zero (un robot "fermo" non ha mai velocità esattamente 0.0), `BLOCKED_AFTER_S=5s` è abbastanza lungo da non scambiare una breve rotazione di recovery di `move_base` per un blocco vero, ma abbastanza corto da restare un precursore utile rispetto alla finestra di 20s del rilevatore di deadlock (§2.6).

I canali di salute (`battery_pct` , `motor_current` , `motor_temp`) non vengono letti da nessun topic ROS: sono sintetizzati, come già detto sopra, perché il TurtleBot3 simulato non li possiede.

Un ultimo dettaglio strutturale: la frequenza di pubblicazione su Kafka (`publish_hz` , rosparam, 2Hz di default) è indipendente dalla frequenza con cui Gazebo/ `move_base` aggiornano davvero odometria e lidar (molto più alta) — un `rospy.Rate` nel loop principale campiona lo stato corrente a un ritmo scelto, non pubblica ogni singolo aggiornamento fisico. Questo disaccoppia deliberatamente il volume dati verso Kafka dalla fedeltà della simulazione fisica: si può alzare o abbassare il carico sulla pipeline senza toccare Gazebo.

Perché JSON e non un formato binario con schema (Avro, Protobuf). Scelta dichiaratamente pragmatica, non un default inconsapevole: JSON è leggibile da un consumer generico (utile in debug, es. il pannello streaming live di §2.11) senza bisogno di un registro schemi condiviso fra Python (`rospy / confluent_kafka`), PySpark e Node.js (`kafkajs`) — tre stack diversi che altrimenti richiederebbero generare e tenere allineate classi da un `.avsc / .proto` comune. Il costo è reale e va dichiarato: payload più grandi e nessuna verifica di schema imposta dal broker (un producer può scrivere un JSON malformato senza che nessuno se ne accorga finché un consumer non prova a deserializzarlo) — accettabile ai volumi di questo progetto (fino a decine di migliaia di messaggi al secondo, §3.2), lo sarebbe assai meno a scala industriale.

2.3 Contratto dati: schema del messaggio di telemetria

Lo schema del messaggio di telemetria è la fonte di verità unica dell'intero progetto: ogni componente (ponte ROS, generatore sintetico, job Spark, storage, layer TAG) si conforma ad esso. Un tick per robot, pubblicato in formato JSON sul topic Kafka `telemetry` :

```
{
  "ts": 1721400000000,
  "robot_id": "R3",
  "x": 12.4, "y": 3.1, "theta": 1.57,
  "v_lin": 0.22, "v_ang": 0.0,
  "cmd_v_lin": 0.25, "cmd_v_ang": 0.0,
  "battery_pct": 74.0, "motor_current": 1.8, "motor_temp": 41.5,
  "min_obstacle_dist": 0.9,
  "task_state": "moving",
  "current_edge": "C-F",
  "goal_node": "H"
}
```

`task_state` assume uno fra `idle` , `moving` , `blocked` , `charging` — il valore `blocked` richiede che *sia* la velocità lineare *sia* quella angolare restino sotto una soglia epsilon per oltre 5 secondi con un goal attivo (una rotazione pura di recovery, tipica di `move_base` , resta quindi classificata `moving` , non `blocked` — un

dettaglio rilevante per il rilevatore di livelock, §2.6). Posa, velocità e `min_obstacle_dist` provengono da Gazebo (odometria e lidar reali); i canali di salute sono sintetizzati come descritto sopra.

La mappa del magazzino (`config/warehouse_graph.json`) è un grafo con nodi (id, coordinate, tipo) e archi (id, estremi, capacità, lunghezza); i robot la seguono come roadmap, e la posizione viene mappata sull'arco occupato. Un secondo file, `config/experiment.json`, definisce la flotta, i percorsi (`goal_sequence` per robot), gli scenari di conflitto e il `fault_schedule` — quest'ultimo è anche, letteralmente, la **ground truth** usata per calcolare precision/recall nella valutazione sperimentale (§3).

2.4 Ingestion: Apache Kafka

Un singolo broker Kafka fa da spina dorsale dell'ingestion. Quattro topic principali:

- **telemetry** — un messaggio per tick per robot, con chiave di produzione `robot_id`: garantisce che i messaggi dello stesso robot siano sempre consegnati nell'ordine in cui sono stati prodotti (proprietà che Kafka garantisce *per chiave*, non globalmente).
- **injected_faults** — un record per ogni guasto iniettato (inizio/fine reali, parametri) — la ground truth sperimentale.
- **anomalies** — un evento per ogni anomalia rilevata dal job di detection (salute, deadlock, livelock).
- **fleet_state** — lo stato arricchito di ciascun robot ad ogni tick (telemetria + flag di anomalia), consumato in tempo reale dalla dashboard.

Scelte implementative e perché

- **KRaft, non Zookeeper.** Il broker (`apache/kafka:3.7.0`) gira in modalità KRaft (`KAFKA_PROCESS_ROLES=broker,controller` sullo stesso nodo, `CLUSTER_ID` fisso): Zookeeper è ormai il percorso deprecato per nuovi cluster Kafka, e per un singolo broker locale KRaft elimina un intero servizio da orchestrare senza perdere nulla di rilevante per questo progetto (replica/quorum multi-nodo non sono un requisito qui).
- **Chiave di produzione `robot_id`, non round-robin.** Scelta esplicita per due motivi: garantisce l'ordinamento causale dei messaggi di uno stesso robot (rilevante per la detection, che ragiona su sequenze temporali per robot — es. il rilevatore di livelock, §2.6), e predispone il sistema a scalare orizzontalmente aumentando il numero di partizioni in futuro, senza dover cambiare né i producer (`kafka_bridge.py`, il generatore sintetico) né i consumer.
- **Un solo broker, una sola partizione per topic nella configurazione attuale.** Va detto con la stessa onestà con cui è dichiarato altrove in questo documento (§4.2): nessuno script del progetto crea esplicitamente i topic con un numero di partizioni maggiore di 1, quindi Kafka li crea automaticamente con il default del broker (1 partizione). La scelta della chiave `robot_id` resta corretta e utile — l'ordinamento per robot è comunque garantito, ed è la premessa necessaria per poter aumentare le partizioni senza modificare il codice — ma il parallelismo di lettura fra robot diversi che una partizione singola per robot avrebbe permesso **non è ancora sfruttato** in questa configurazione. Con il volume reale del progetto (poche decine di messaggi al secondo dalla flotta ROS, fino a decine di migliaia dal generatore sintetico, §2.5) un singolo broker a partizione singola non è mai risultato il collo di bottiglia negli esperimenti di efficiency (§3.2) — il limite osservato è altrove (il generatore stesso, o la CPU condivisa fra Spark e Gazebo, §3.3) — ma è un margine di scalabilità orizzontale lasciato deliberatamente sul tavolo, non sfruttato perché non necessario finora.
- **Nessun limite di risorse (CPU/RAM) impostato su nessun container** in `docker-compose.yml`, Kafka incluso: scelta pragmatica per un ambiente di sviluppo/demo su una singola macchina, dove i container competono liberamente per le risorse dell'host — rilevante anche per la discussione sulla scalabilità in §3.3.

2.5 Due modalità di simulazione: flotta reale e generatore sintetico

Il progetto produce telemetria in due modi indipendenti che condividono *esattamente* lo stesso schema (§2.3) e la stessa pipeline a valle (Kafka → Spark → storage/dashboard): nessun componente successivo a `kafka_bridge.py/synthetic_generator.py` sa distinguere l'origine di un messaggio. Non sono due sistemi alternativi, ma due sorgenti compatibili pensate per scopi diversi — vale la pena spiegare come funziona ciascuna e, soprattutto, quando ha senso usare l'una piuttosto che l'altra.

Come funziona la flotta reale

Descritta in dettaglio in §2.2: ogni robot è un TurtleBot3 fisicamente simulato in Gazebo (fisica, lidar, dinamica reali), guidato da `move_base` lungo una sequenza di goal fissa (`goal_sequence`, letta da `config/experiment.json`); i canali di salute sono sintetizzati nel ponte `kafka_bridge.py` a partire da parametri nominali, ma la posa e la velocità sono quelle vere restituite dal motore fisico. I conflitti di traffico (deadlock, livelock) nascono da una contesa reale su corridoi fisicamente stretti, non da una regola scritta a mano.

Come funziona il generatore sintetico

Gazebo non arriva a volumi di decine di migliaia di messaggi al secondo (il collo di bottiglia è la simulazione fisica, non Kafka/Spark). Per gli esperimenti di scalabilità (§3.2) è stato costruito un generatore puro Python (`generator/synthetic_generator.py`, nessuna dipendenza da ROS) che produce robot-token: percorrono lo stesso grafo del magazzino arco per arco a velocità costante, scegliendo il prossimo nodo **a caso** fra i vicini (nessuna sequenza predisposta, a differenza dei robot ROS), e pubblicano messaggi che rispettano *esattamente* lo stesso schema di `kafka_bridge.py`. Il carico è controllato da due parametri indipendenti (numero di robot-token × frequenza per robot); una coda a priorità (`heapq`) permette di scalare la schedulazione a migliaia di sorgenti in un solo processo. Il generatore supporta anche l'iniezione di guasti con lo stesso formato e la stessa logica di applicazione di `kafka_bridge.py`, così le metriche di precision/recall si calcolano nello stesso modo sia sui dati reali sia su quelli sintetici — con l'eccezione dei tre nodi dedicati alla flotta reale (`DEPOSITO/RISERVA1/RISERVA2`, §2.11), esclusi dal random walk perché non hanno senso per un robot-token generico.

Il modello dei dati: non un replay di bag ROS

Vale la pena essere espliciti su un punto non ovvio dal solo nome "generatore sintetico": non è un replay di una registrazione reale (una `rosbag` o uno storico Parquet campionato), ma una simulazione parametrica costruita da zero, pensata per essere statisticamente plausibile senza dipendere da nessun dato osservato in

precedenza. Nello specifico:

- **Movimento:** `VirtualRobot.tick()` calcola la posizione per interpolazione lineare lungo l'arco corrente a velocità costante (`speed_mps` , parametro libero) — non fisica, non un modello cinematico, solo geometria ($x = x_1 + t \cdot (x_2 - x_1)$, $t = \text{distanza percorsa} / \text{lunghezza arco}$). Il prossimo nodo è scelto a caso fra i vicini del grafo, evitando di tornare subito indietro se esistono alternative.
- **Schedulazione:** un solo processo Python gestisce migliaia di robot-token tramite una coda a priorità (`heapq`) di eventi "prossimo tick dovuto" — non un thread o una coroutine per robot (che non scalerebbe a migliaia di sorgenti), un min-heap che estrae sempre l'evento più imminente, gli fa produrre un messaggio, e lo re-inserisce per il tick successivo (`due_ts + periodo`).
- **Canali di salute:** stessa formula e stessi parametri di `kafka_bridge.py` — rumore gaussiano attorno a un nominale letto da `health_channels_nominal` in `config/experiment.json` , non una distribuzione inventata a parte per il generatore. È una scelta deliberata di coerenza: un guasto iniettato deve "spiccare" nello stesso modo sopra lo stesso rumore di fondo, sia sulla flotta reale sia su quella sintetica, altrimenti precision/recall non sarebbero confrontabili fra le due sorgenti.
- **Guasti:** stessa classe `FaultInjector` , stesso `fault_schedule` , stesse firme parametriche di §2.2 — condivise per file (non duplicate) proprio perché generatore e ponte ROS devono produrre un guasto indistinguibile a valle.

In sintesi: il generatore non "ricorda" nulla di una run precedente né riproduce una traccia registrata — ogni esecuzione è un nuovo campionamento pseudo-casuale (seed opzionale, per riproducibilità nei test) da un modello statistico dichiarato, non un replay.

Quale usare per i test, e perché

La scelta non è indifferente: ciascuna modalità rende osservabile un aspetto diverso del sistema, e forzarla sull'altra o non produce il dato cercato o lo produce con troppa variabilità per essere utile.

Cosa si vuole misurare	Modalità	Perché
Throughput a carichi crescenti, punto di rottura	Generatore sintetico	Gazebo non raggiunge questi volumi: il limite sarebbe la fisica simulata, non la pipeline dati (§3.3)
Precision/recall della detection, accuratezza della previsione (offline e live), execution accuracy del TAG	Generatore sintetico	Serve ripetibilità e controllo esatto su quali/quantı robot hanno un guasto noto in una finestra breve; la fisica di Gazebo introduce variabilità che non aiuta a isolare la metrica
Conflitti di traffico "veri" (deadlock/livelock)	Flotta reale	Solo Gazebo produce una contesa fisica reale su un corridoio stretto; il generatore riproduce la logica del grafo ma non una collisione fisica
Anello di self healing (previsione → riparazione preventiva, freeze, decommissioning)	Solo flotta reale	Deliberatamente non cablato sul generatore (§2.11): la guardia <code>REAL_ROBOT_ID_RE</code> in <code>fleetStateStore.js</code> ignora esplicitamente i robot-token sintetici — il generatore è pensato per robot intercambiabili in un test di carico, non per la nozione di "missione propria" che l'anello richiede
Interrogazione in linguaggio naturale (TAG)	Indifferente	Entrambe le sorgenti scrivono lo stesso storico Parquet a valle; il TAG interroga lo storico, non la sorgente

Questa distinzione è la ragione per cui `eval/` è organizzato così: `run_effectiveness.py` e quasi tutto `run_efficiency.py` (throughput, latenza onset→alert, detection sotto carico crescente) girano sempre sul generatore sintetico; un esperimento dedicato di reattività (latenza fra la previsione rilevata in streaming e la riparazione effettivamente dispiacciata) richiede invece esplicitamente la flotta reale già avviata, e si salta da solo — senza far fallire il resto del run — se non lo è.

2.6 Speed layer: detection real-time in Spark Structured Streaming

Il cuore del sistema di detection è `streaming/detection_job.py` , un solo processo `spark-submit` persistente. Legge il topic Kafka `telemetry` **una sola volta** (`startingOffsets=latest` , un solo `readStream`), lo arricchisce con la distanza-sul-grafo verso il goal (Floyd-Warshall precalcolato, distribuito via `broadcast`), e da quell'unico DataFrame derivano **quattro query Structured Streaming indipendenti**, ciascuna col proprio sink, checkpoint e intervallo di trigger — separarle, invece di una query monolitica, mantiene semplice una logica che ha semantiche radicalmente diverse (per-messaggio contro aggregazioni a finestra contro stato esplicito per chiave):

Query	Trigger	Semantica	Scrivi su
Salute	2s	per-messaggio	<code>fleet_state</code> , <code>anomalies (type=salute)</code>
Deadlock	10s	finestra scorrevole 20s	<code>anomalies (type=deadlock)</code>
Livelock	10s	stato esplicito per <code>robot_id</code> (<code>applyInPandasWithState</code>)	<code>anomalies (type=livelock)</code>
Previsione	5s	stato esplicito per (<code>robot_id</code> , canale)	<code>anomalies (type=previsione)</code>

Detection di salute

Trigger a 2 secondi, per ogni messaggio: due tecniche complementari applicate in parallelo.

- **Soglie statiche:** `motor_temp > 55°C` , `motor_current > 2.5A` , `battery_pct < 20%` — deterministiche, calibrabili offline (soglie adattive, §2.7).
- **Isolation Forest** (scikit-learn), applicato via `pandas_udf` sul vettore (`motor_temp` , `motor_current` , `battery_pct` , `v_lin` , `min_obstacle_dist`). Allenato *offline* su un campione sintetico di telemetria nominale generato dagli stessi parametri di `config/experiment.json` (coerenza fra chi genera i dati nominali e chi allena il modello), non online sui dati che sta giudicando — un modello che si auto-allena sui dati che valuta rischierebbe di "abituarsi" ai guasti stessi. Il campionamento sintetico modella esplicitamente la correlazione fra stato di moto e prossimità agli ostacoli (un robot fermo resta

"parcheggiato" e la sua distanza da un ostacolo è uno stato stabile per l'intera sosta, non un evento raro — dettaglio emerso e corretto durante l'estensione Passo 14, §2.11).

Ogni messaggio arricchito viene scritto su `fleet_state` (stato live per la dashboard); se una delle due tecniche segnala un'anomalia, viene scritto anche su `anomalies` con `type="salute"` e l'elenco delle ragioni (`threshold_reasons`, eventualmente vuoto se solo l'Isolation Forest ha reagito). Verificato empiricamente che le due tecniche sono complementari, non ridondanti: in un test mirato una soglia su `motor_temp` scattava mentre l'Isolation Forest restava silenzioso (temperatura isolata alta, ma il resto del vettore "sembrava" nominale), e il pattern si è parzialmente invertito su uno spike di corrente.

Nota per chi legge in sequenza: entrambe le tecniche restano sempre attive per la detection descritta qui — *non* sono state rimosse in nessun punto del progetto. In §2.11 (estensione Passo 14) l'Isolation Forest viene *disaccoppiata* da un solo, specifico consumatore a valle — l'azione automatica di self healing sulla flotta reale, per un problema di falsi positivi statistici diventati consequenziali una volta collegati a un'azione reale — non dalla pipeline di detection in sé, dove continua a contribuire come qui descritto.

Come è allenato l'Isolation Forest, in pratica

`streaming/isolation_forest_model.py` — `train_model()` genera 5000 vettori sintetici a 5 feature (`motor_temp`, `motor_current`, `battery_pct`, `v_lin`, `min_obstacle_dist`, seed fisso 42 per riproducibilità) e allena `sklearn.ensemble.IsolationForest(n_estimators=100, contamination=0.02)`. Non è un campionamento indipendente feature-per-feature: `v_lin` e `min_obstacle_dist` sono generati insieme, condizionati su uno stato "in movimento / fermo" scelto prima (60% probabilità di movimento) — un robot fermo ha una probabilità del 50% di essere "parcheggiato" vicino a qualcosa (`min_obstacle_dist` fra 0.3 e 2.0m, non un evento raro) invece che in area aperta, e un robot in movimento ha invece il 70% di probabilità di trovarsi in area aperta. Questa correlazione esplicita non è un dettaglio cosmetico: un primo campionamento indipendente delle due feature sotto-rappresentava la combinazione "fermo e vicino a qualcosa", causando falsi positivi sistematici quando un robot restava parcheggiato a lungo vicino a uno scaffale (§2.11, Passo 14) — il modello non l'aveva mai visto abbastanza spesso in allenamento per considerarlo normale. Il modello allenato viene serializzato (`pickle`) su file e ricaricato agli avvisi successivi di `detection_job.py`, con un fallback che lo riallena al volo se il file manca — mai un motivo per bloccare l'avvio del job.

Rilevamento del deadlock

Finestra scorrevole di 20 secondi (slide 10s) per `current_edge`: se nella finestra compaiono almeno due robot distinti con `task_state="blocked"` sullo stesso arco, viene emessa un'anomalia `type="deadlock"`.

Rilevamento del livelock: un caso di studio sulla semantica dello streaming

Il rilevatore di livelock ha attraversato tre iterazioni successive, ciascuna motivata da un falso positivo reale osservato in dashboard, non da un'analisi teorica preventiva — un esempio istruttivo di quanto le semantiche di windowing di Structured Streaming siano insidiose:

- Versione iniziale:** finestra scorrevole di 30s, anomalia se la distanza-sul-grafo dal `goal_node` non cala. La distanza era però agganciata al *nodo più vicino*, non calcolata in modo continuo lungo l'arco: per circa metà del tempo di attraversamento di un arco lungo (fino a 75s a velocità di crociera), la distanza restava perfettamente costante anche con moto reale continuo, generando decine di falsi positivi. **Fix:** distanza calcolata proiettando `(x,y)` sull'arco corrente e sommando la distanza-sul-grafo (precalcolata con Floyd-Warshall su tutto il grafo, distribuita via `broadcast`) dal punto proiettato al goal.
- Secondo bug, indipendente,** trovato dalla suite di test: le query usavano `outputMode("update")`, che riemette una riga ogni volta che l'aggregato di una finestra *cambia*, anche quando la finestra è ancora aperta e parzialmente popolata — un robot osservato nei primi secondi di una nuova finestra (pochi messaggi, poco spostamento visibile *finora*) poteva soddisfare "nessun progresso" su un campione incompleto. **Fix:** `outputMode("append")`, che emette solo dopo che il watermark ha chiuso la finestra per intero (costo: più latenza, fino a un'altra finestra completa).
- Terzo bug:** anche con una finestra chiusa per intero, il rilevatore restava una valutazione *a scatto singolo* su 30 secondi — un breve accodamento durante un sorpasso su un corridoio a corsia singola rientra facilmente in una finestra con poco progresso netto, pur essendo un evento normale che si risolve da solo. Un primo tentativo di fix (due aggregazioni a finestra incatenate, per richiedere almeno due finestre candidate consecutive) si è rivelato rompersi su un limite noto di Spark: il watermark del secondo stadio restava bloccato all'epoch nonostante dati reali in arrivo. **Fix definitivo:** stato esplicito per `robot_id` via `applyInPandasWithState` — un "checkpoint" di riferimento (distanza, istante) aggiornato ogni 10s di event time; se il progresso fra un checkpoint e il successivo è sotto soglia si accumula un contatore di stallo continuo, altrimenti si azzera; l'anomalia scatta solo dopo 60 secondi *consecutivi* di stallo. Questa versione implementa correttamente la definizione di "livelock" come condizione *prolungata*, non un singolo campionamento sfortunato.

Il rilevatore finale è quindi un vero operatore stateful per chiave, non un'aggregazione a finestra — più pesante computazionalmente (una chiamata Python per robot per micro-batch), ma l'unico che modella correttamente la semantica richiesta.

Quarto bug, trovato più tardi (Passo 15), usando il generatore sintetico invece della flotta reale. Il generatore sintetico produceva decine di falsi positivi di livelock in pochi minuti, la flotta reale quasi mai. Causa: il generatore assegna un nuovo `goal_node` a caso ogni volta che un robot arriva a destinazione, senza mai fermarsi, mentre un robot ROS reale segue una sequenza di goal fissa e finita; se il `goal_node` cambia fra un campionamento di riferimento e il successivo, la distanza verso il vecchio obiettivo (quasi raggiunto) viene confrontata con quella verso il nuovo (appena assegnato) — sembra un arretramento netto anche se il robot si è mosso normalmente. Con archi fisicamente lunghi rispetto all'intervallo di campionamento (10s) un singolo cambio di goal non basta a far scattare l'anomalia da solo, ma il generatore espone `speed_mps` come parametro libero e sganciato dalla fisica: a velocità elevate un arco si percorre in meno tempo di un campionamento, e quasi ogni campionamento rischia di cadere a cavallo di un cambio goal. **Fix:** lo stato del rilevatore tiene ora anche il `goal_node` del campionamento di riferimento; se cambia, la riga successiva è trattata come un nuovo riferimento invece che confrontata contro un obiettivo diverso. Verificato: generatore a `speed_mps=1.0` per 130s (lo scenario che prima produceva lo sciame) → zero eventi; i test di vero/falso positivo del livelock e del deadlock continuano a passare dopo il fix.

Previsione in streaming: la quarta query

Stesso pattern del livelock (stato esplicito per chiave via `applyInPandasWithState`), ma la chiave qui è (`robot_id`, `channel`) e lo scopo è diverso: non rilevare un'anomalia già in corso, ma un **preavviso**. Per ciascun canale di salute, una soglia "morbida" è derivata da quella dura con un margine fisso (`motor_temp` -10°C, `motor_current` -0.5A, `battery_pct` +10%). Lo stato tiene, per ogni coppia (robot, canale), i timestamp (event time) degli ultimi superamenti della soglia

morbida in una finestra scorrevole di 60 secondi; al terzo superamento nella finestra scatta un'anomalia `type="previsione"`, con un `lead_time_s` stimato in modo *conservativo* pari alla finestra stessa (60s) — non una regressione come quella offline (§2.8), perché un segnale a raffiche non si presta a un fit lineare stabile. Lo stato scade dopo 90s di inattività (`state.setTimeoutTimestamp`).

Il caso d'uso primario è un nuovo tipo di guasto, `preavviso_intermittente` (§2.2): raffiche saltuarie che tornano al valore nominale fra una raffica e l'altra, quindi invisibili a una soglia fissa continua. Verificato empiricamente che il meccanismo generalizza oltre questo caso: anche i guasti a rampa continua già esistenti (`deriva_termica`, `spike_corrente`) attraversano la soglia morbida prima di quella dura, quindi vengono anticipati da una previsione anche loro, per costruzione, non solo dal nuovo guasto a raffiche. La reazione applicativa che questa previsione attiva sulla flotta reale (riparazione preventiva, dispaccio riserva) è descritta in §2.11.

Rischio, anomalia, danno: tre misure distinte

Il progetto misura tre cose concettualmente diverse, spesso confuse fra loro in una lettura superficiale, ma tenute deliberatamente separate nella pipeline:

Misura	Cos'è	Dove si trova	Quando è disponibile
Danno (ground truth)	Un guasto realmente avvenuto — iniettato per costruire un esperimento controllato (§2.2), o (in linea di principio) un guasto reale del hardware	topic/tabella <code>injected_faults</code> : <code>robot_id</code> , <code>fault_type</code> , finestra <code>[start_ts, end_ts]</code> reale	<i>A posteriori</i> rispetto al momento in cui è successo — è la verità con cui si giudicano le altre due misure (§3.1)
Anomalia (detection)	Ciò che il sistema rileva <i>ora</i> , in tempo reale, guardando la telemetria corrente — soglie fisse + Isolation Forest per la salute (sopra), finestre/stato per deadlock e livelock	topic <code>anomalies</code> , <code>type</code> ∈ { <code>salute</code> , <code>deadlock</code> , <code>livelock</code> }	Al momento in cui la condizione anomala è già in corso (con la latenza del trigger, tipicamente pochi secondi per la salute, fino a 60s+ per il livelock, per costruzione — §2.6)
Rischio (previsione)	Una stima di <i>quando</i> un canale di salute supererà una soglia critica, calcolata sul trend storico — non è ancora successo nulla	due fonti indipendenti : topic <code>anomalies</code> con <code>type="previsione"</code> (speed layer, §2.6, <code>lead_time_s</code> stimato in modo conservativo) e tabella <code>predictions</code> (§2.12, batch layer, §2.8, <code>lead_time_s</code> da una regressione vera) — unite nella stessa vista dashboard (§2.10), la live vince se presente perché più fresca	<i>Prima</i> che l'anomalia scatti — è l'unica delle tre misure pensata per essere proattiva, non reattiva (§2.9), e l'unica calcolata da entrambi i layer (§2.1)

La distinzione conta perché le tre misure hanno gradi di certezza radicalmente diversi: il danno è un fatto (o è successo o non è successo), l'anomalia è un giudizio deterministico o statistico su dati già osservati (può sbagliare: falsi positivi/negativi, §3.1), il rischio è un'estrapolazione (può sbagliarsi ancora di più, dato che assume che il trend attuale continui invariato — discusso esplicitamente in §2.9 a proposito del guasto che si "auto-risolve" durante l'esperimento). Il layer TAG (§2.9) può interrogare tutte e tre insieme proprio perché restano tabelle separate con la stessa chiave (`robot_id`, tempo) — es. "quali robot hanno un rischio alto ma nessuna anomalia rilevata ancora?" è una domanda a cui si può rispondere solo avendole tenute distinte.

Infrastruttura Spark: scelte implementative e perché

- **Cluster standalone (master + worker), non YARN/Kubernetes/ `local[*]`**. Standalone è la modalità più semplice che Spark offre per un vero cluster distribuito (driver/master/worker separati) senza bisogno di un resource manager esterno — coerente con un deployment Docker Compose a container fissi, non un ambiente cloud elastico. Rispetto a `local[*]` (un solo processo, usato tipicamente per debug), separare master e worker rispecchia più fedelmente un'architettura Spark di produzione, anche se qui gira su una singola macchina fisica.
- **Immagine Docker custom** (`spark/Dockerfile`, base `bitnamilegacy/spark:3.5.6`) con `pandas` / `numpy` / `pyarrow` / `scikit-learn` aggiunti: servono agli executor per il `pandas_udf` dell'Isolation Forest e per `applyInPandasWithState` del livelock — l'immagine bitnami base ha solo PySpark puro.
- **Connettore Kafka risolto a runtime** (`--packages org.apache.spark:spark-sql-kafka-0-10` via Maven/Ivy a ogni avvio, non incluso nell'immagine), con la cache Ivy su un volume Docker dedicato per non riscaricare le stesse dipendenze a ogni riavvio del container.
- **Checkpoint: `/tmp` per `detection_job.py`, `/data/_checkpoints` (persistente) per `persistence_job.py` (§2.7)**. Non è un'incoerenza ma una scelta deliberata, diversa per scopi diversi: `detection_job.py` legge solo `latest` (vuole reagire a ciò che succede ora, non recuperare uno storico), quindi perdere il checkpoint a un riavvio non ha conseguenze — riparte semplicemente da "adesso". `persistence_job.py` legge `earliest` e costruisce l'archivio storico: perdere il suo checkpoint causerebbe la riletture e la duplicazione di dati già scritti su Parquet, quindi il suo checkpoint vive sul volume persistente `/data`, non in un `/tmp` che sparisce al riavvio del container.
- **`spark.sql.streaming.stateStore.stateSchemaCheck=false`**, solo per `detection_job.py`: il controllo di compatibilità dello schema di stato fra un micro-batch e l'altro dà un falso positivo (`StateSchemaNotCompatible`) su operatori stateful Python (`applyInPandasWithState`) anche a schema invariato — un limite noto di questa combinazione di Spark, non un problema di logica.
- **`spark.cores.max` fissato esplicitamente per ciascun job persistente** (non lasciato al default): lo scheduler FIFO di Spark standalone assegna per default *tutti* i core disponibili al primo job che parte, il che con più job persistenti sullo stesso cluster (detection, query_service del layer TAG, §2.9) affamerebbe gli altri. Il valore esatto per `detection_job.py` è stato rivisto due volte in corso d'opera, per due motivi opposti — un caso concreto di quanto sia difficile tarare una singola costante quando lo stesso job serve scenari di carico molto diversi: **alzato** da 8 a 10 core durante la costruzione della suite di test (carico pesante del generatore sintetico, §2.5, dove i micro-batch restavano indietro); **riportato giù** a 4 core dopo aver scoperto che, sulla flotta ROS reale (poche unità di robot), quegli stessi 10 core mettono Spark in competizione diretta con Gazebo per la CPU della stessa macchina — nessun limite di risorse è impostato sui container (§2.4), quindi la contesa non è arbitrata da nessuno se non dalla configurazione dei singoli job. Dettagli della diagnosi in §3.3.

2.7 Batch layer: persistenza e soglie adattive

Un solo Kafka, due consumatori Spark indipendenti — non un bivio, due letture in parallelo

Vale la pena essere precisi su un punto che i nomi delle due sezioni ("speed layer", §2.6, e questa, "batch layer") potrebbero far fraintendere: non esiste un bivio dove i dati "vanno verso Spark Streaming oppure verso Parquet". `detection_job.py` e `persistence_job.py` sono **due processi spark-submit separati**, entrambi Spark Structured Streaming (stessa tecnologia, §2.1), che leggono *gli stessi* topic Kafka **in parallelo** e in modo del tutto indipendente — Kafka non smista nulla fra i due, ogni consumer group legge la propria copia indipendente del log. Cambia solo cosa ciascuno ne fa:

- `detection_job.py` legge `startingOffsets=latest` (gli interessa solo l'adesso) e **scrive di nuovo su Kafka** (`fleet_state`, `anomalies`): un output effimero, pensato per essere consumato subito dal backend, mai per l'archivio.
- `persistence_job.py` legge `startingOffsets=earliest` (gli interessa tutto lo storico) e **scrive su Parquet**: un output durevole, interrogato molto più tardi da Spark SQL (`query_service.py`, §2.9), da `forecast_failures.py` (§2.8) o dagli script di valutazione sperimentale (§3).

Anche il *modo* di scrivere differisce, e non per caso. `persistence_job.py` usa il sink nativo `writeStream.format("parquet")`: una sola destinazione per query, Spark gestisce da solo la scrittura dei file e il commit del checkpoint. `detection_job.py` avvolge invece la scrittura in `foreachBatch` (funzione `to_kafka()`, §2.6) perché il caso più importante — la query di salute — deve scrivere **due** topic diversi a partire dalla stessa elaborazione (`fleet_state` sempre, `anomalies` solo se `health_anomaly` è vero): nessun sink nativo di Structured Streaming supporta un fan-out del genere, va scritto a mano un batch alla volta.

`streaming/persistence_job.py` — tre query Structured Streaming indipendenti (stesso pattern del job di detection) che leggono `telemetry`, `anomalies`, `injected_faults` con `startingOffsets=earliest` (a differenza del job di detection, che legge solo `latest`: qui lo scopo è costruire l'archivio storico completo) e li scrivono su **Parquet**, su un volume Docker condiviso montato in sola lettura dal backend. `anomalies` è partizionato per `type`, dato che le query a valle (TAG, valutazione sperimentale) filtrano quasi sempre per tipo.

`offline/adaptive_thresholds.py` chiude un anello di feedback verso la detection: usa `injected_faults` come **ground truth** per classificare ogni anomalia di salute storica come vero o falso positivo, e per i canali con un numero minimo di falsi positivi ricalcola la soglia come il 99.5° percentile (0.5° per la batteria) dei valori osservati nei periodi *nominali* (fuori da ogni finestra di guasto reale) — non un numero scelto a mano, ma calibrato sul rumore nominale realmente osservato. Il job di detection legge queste soglie ricalibrate all'avvio (non un hot-reload a caldo su una query già in esecuzione: il feedback avviene fra una run e la successiva).

2.8 Analisi predittiva offline (time series)

`predictive/forecast_failures.py` stima, per ogni coppia (robot, canale di salute), *quando* il valore supererà la soglia critica del guasto corrispondente — non solo se è già in anomalia, ma con quanto anticipo (*remaining useful life*). La pipeline, in batch su pandas (nessun overhead di una `SparkSession` per questo volume di dati):

1. Ricampiona a bin di 5s la finestra recente di storico (default: ultimi 300s).
2. **Pre-filtro sulla pendenza**: se la variazione per minuto è sotto una soglia minima calibrata sul rumore nominale, si ferma qui — evita fit spurii e falsi allarmi su rumore che imita per caso un trend.
3. Solo in presenza di un trend vero, allena una **regressione lineare ai minimi quadrati** (`numpy.polyfit`, grado 1) e calcola *analiticamente* l'istante di incrocio con la soglia critica ($t = (\text{soglia} - \text{intercetta}) / \text{pendenza}$) — nessun bisogno di generare un forecast punto-per-punto.
4. Le soglie critiche coincidono deliberatamente con i valori-obiettivo delle firme di guasto (§2.2): 85°C è il `plateau_temp_c` di `deriva_termica`, 4.5A è il `peak_a` di `spike_corrente`.

La scelta della tecnica è stata rivista in corso d'opera: la prima versione usava **ARIMA** (statsmodels), come indicato dal piano originale. Un problema concreto (il default `trend='n'` di `ARIMA(p,1,0)` fa convergere il forecast a un livello asintotico costante invece di continuare a estrapolare una rampa) e la constatazione che il segnale è quasi lineare per costruzione (le firme di guasto sono rampe lineari verso un plateau) hanno motivato il passaggio a una regressione lineare — tecnica più semplice, più facile da verificare, con risultati praticamente identici ad ARIMA sullo stesso segnale reale (130.5s contro 130.0s di lead time misurato).

2.9 Serving layer: Spark SQL e sistema TAG con Hugging Face

Il quarto e ultimo requisito tecnologico obbligatorio — un LLM — è implementato come layer **TAG (Table-Augmented Generation)**, secondo la definizione a tre stadi del framework omonimo (Biswal et al., "TAG: A Unified Framework for Table-Augmented Generation", 2024): (1) *query synthesis* — la domanda in linguaggio naturale diventa una query eseguibile; (2) *query execution* — la query gira sui dati storici Parquet; (3) *answer synthesis* — il risultato grezzo viene rielaborato dall'LLM in una risposta in linguaggio naturale. Il terzo stadio è quello che distingue un sistema TAG da un semplice text-to-SQL (che si ferma al secondo, restituendo righe grezze) — è stato aggiunto in un secondo momento proprio per questo motivo, non presente nella prima versione del layer (vedi nota più sotto).

Esecuzione SQL: Spark SQL, non un motore embedded

Il piano originale prevedeva DuckDB (libreria Node embeddable). È stata preferita una soluzione che riusa il cluster Spark già in piedi per la detection e la persistenza: `streaming/query_service.py`, un processo Spark **persistente** con un server HTTP nativo (`http.server`, nessuna dipendenza aggiuntiva per due sole route) che ricrea le temp view sulle quattro cartelle Parquet ad ogni richiesta (economico: è solo discovery dello schema, non un'azione, quindi vede sempre i dati più recenti scritti dal batch layer) ed esegue la query con un `LIMIT` di sicurezza. Evitare `spark-submit` per ogni domanda risparmia i circa 10 secondi di avvio JVM per query, inaccettabili per un endpoint interattivo.

Generazione SQL: Qwen2.5-Coder-32B via router Hugging Face

Il modello — `Qwen/Qwen2.5-Coder-32B-Instruct` — non è ospitato in locale: viene chiamato tramite il router **Inference Providers** di Hugging Face (`https://router.huggingface.co/v1/chat/completions`, API compatibile OpenAI, autenticazione Bearer). Questa scelta evita di dover gestire GPU/quantizzazione/`nvidia-container-toolkit` nel container per un modello da 32 miliardi di parametri, mantenendo comunque un LLM realmente open-source (non un servizio proprietario chiuso).

Pipeline dell'endpoint `POST /api/tag`

Backend Node.js, quattro moduli con responsabilità separate:

- `promptBuilder.js` — costruisce *due* prompt distinti, uno per stadio: `buildMessages` per la query synthesis (schema completo delle quattro tabelle Parquet, con le peculiarità del dialetto Spark SQL rilevanti — accesso a `STRUCT` con dot notation, `array_contains` per campi ARRAY, `timestamp_millis` per i timestamp epoch — 5 esempi few-shot, regole esplicite: solo `SELECT`, nessuna spiegazione testuale, `LIMIT` ragionevole se non specificato); `buildAnswerSynthesisMessages` per l'answer synthesis (domanda + SQL eseguita + righe risultato in JSON, con l'istruzione esplicita di rispondere solo in base a quei dati e di dichiarare "nessun risultato" invece di inventare — mitigazione diretta del rischio di allucinazione tipico di questo stadio).
- `LlmService.js` — client verso il router Hugging Face, riusato identico per entrambi gli stadi (stesso modello, stesso token: nessuna infrastruttura aggiuntiva per l'answer synthesis).
- `sqlGuard.js` — *difesa in profondità*, solo sullo stadio di query synthesis: ripulisce eventuali blocchi markdown residui e valida che la query generata sia una singola `SELECT / WITH`, priva di parole chiave di scrittura (con controllo a confini di parola, per non bloccare per errore identificatori come `created_at` a causa della sottostringa "creat"), **sia lato Node sia lato `query_service.py`** — due controlli indipendenti sullo stesso vincolo.
- `tagService.js` — orchestra i tre stadi: prompt → LLM → guardia → esecuzione (con **un retry** automatico, ri-prompting il modello con l'errore ricevuto, se la guardia rifiuta la query o l'esecuzione fallisce) → una seconda chiamata LLM che sintetizza la risposta dalle righe ottenute. Se questa seconda chiamata fallisce (es. rate limit del router), l'errore è isolato e non fa fallire la richiesta: l'utente riceve comunque SQL e righe, solo senza la frase in linguaggio naturale — degrado educato, non un errore bloccante.

Verificato che il modello generalizza oltre gli esempi few-shot forniti allo stadio di query synthesis: nessuno dei 5 esempi nel prompt usa una `JOIN` o più di una CTE, eppure il sistema ha generato correttamente una query con CTE e `JOIN` per una domanda che lo richiedeva (§3.1).

Nota di processo — da text-to-SQL a TAG. La prima versione di questo layer (Passo 10 del piano originale) si fermava alla query execution: restituiva SQL generato e righe grezze in una tabella HTML, senza alcuna rielaborazione — un sistema di *text-to-SQL*, non di *TAG* nella definizione stretta del framework, nonostante fosse già così chiamato nella documentazione di progetto (imprecisione poi corretta). Lo stadio di answer synthesis è stato aggiunto in un secondo momento, quando la domanda "è corretto chiamarlo TAG?" ha reso esplicito il divario fra nome e implementazione. L'aggiunta ha richiesto poco lavoro proprio perché l'infrastruttura per il secondo stadio (client HTTP verso il router, credenziali, gestione errori) era già completamente in piedi: è bastata una funzione di prompt in più e una chiamata in più allo stesso `LlmService.js`, riusando lo stesso modello — nessun nuovo servizio, nessuna nuova credenziale.

2.10 Presentazione: backend Node.js e dashboard

Il backend (Node.js/Express) consuma `fleet_state` da Kafka (client `kafkajs`) e fa push via WebSocket ai client connessi; espone inoltre le route REST per il layer TAG, le previsioni, il controllo del generatore sintetico, il controllo della flotta reale e i risultati sperimentali. La dashboard (HTML + canvas 2D, JavaScript puro, nessun framework) disegna la mappa del magazzino, i robot colorati per stato con un anello di segnalazione per le anomalie, gli archi a capacità singola tratteggiati. Per un'animazione fluida nonostante il micro-batch di detection arrivi al massimo ogni 2 secondi, il client applica *dead reckoning*: estrapola la posizione ad ogni frame usando la velocità nota dall'ultimo messaggio, invece di attendere il prossimo pacchetto.

Fra i pannelli disponibili: stato di tutti i robot (reali e sintetici) in tabella, log degli eventi comportamentali (deadlock/livelock) in tempo reale, pannello previsioni (dal layer predittivo), casella di interrogazione in linguaggio naturale, pannello di controllo per avviare/fermare il generatore sintetico scegliendo topologia e guasti, e — nell'estensione più recente — un pannello che lancia gli esperimenti di valutazione sperimentale *on demand* mostrandone i risultati mano a mano che ciascun sotto-esperimento termina, invece di un refresh periodico su grafici pre-generati.

Protocollo WebSocket: cinque tipi di messaggio, un solo endpoint

Un solo endpoint (`/ws`) e un solo `WebSocketServer` lato backend: nessuna sottoscrizione per-topic lato client, ogni messaggio viene diffuso (*broadcast*) a tutti i client connessi, che filtrano lato loro in base al campo `type`. Cinque tipi, ciascuno alimentato da una sorgente diversa:

Type	Quando viene inviato	Sorgente
snapshot	una volta, alla connessione di un nuovo client	<code>fleetStateStore.getSnapshot()</code> — lo stato corrente di tutti i robot noti, così un client che si collega a metà sessione non parte da una mappa vuota
update	ad ogni nuovo <code>fleet_state</code> consumato da Kafka	<code>fleetStateStore.onUpdate()</code>
remove	quando un robot sparisce (timeout, decommissioning, nuovo run)	<code>fleetStateStore.onRemove()</code>
anomaly	ad ogni evento sul topic <code>anomalies</code>	<code>anomalyStream.onEvent()</code>
raw	campionato, al più ogni ~200ms per topic (§2.11, Passo 16)	<code>rawStream.onRawMessage()</code> — pannello streaming live, debug

Nessuna logica di filtro o aggregazione lato WebSocket: i tre servizi (`fleetStateStore.js`, `anomalyStream.js`, `rawStream.js`) sono consumer Kafka `kafkajs` indipendenti (§2.13), ciascuno con la propria logica di trasformazione a monte; il livello WebSocket si limita a serializzare e spedire, la stessa separazione delle responsabilità già vista nell'architettura generale del backend.

2.11 Estensione: self healing sulla flotta reale (e previsione live, Passo 15/16)

Oltre ai 13 passi del piano originale, in vista di una dimostrazione dal vivo è stato realizzato un primo anello di retroazione reale limitato alla flotta ROS/Gazebo. Fino a questo punto il sistema era puramente diagnostico (rilevare, prevedere, segnalare — mai agire su ROS), coerentemente con il principio "il robot è solo la

sorgente dati". L'estensione introduce un'eccezione deliberata e documentata, non una deriva silenziosa dello scope.

- **Corridoi fisici**: i due archi a corsia singola del grafo sono stati fisicamente ristretti in Gazebo con pareti statiche, rendendo osservabili su dati reali (non solo sintetici) sia il deadlock sia il livelock.
- **Iniezione di guasti dal vivo**: un nuovo canale di controllo ROS (`-fault_inject` , JSON su `std_msgs/String`) permette di aggiungere un guasto al bridge *a runtime* — riusa esattamente la stessa logica di attivazione/applicazione/logging dei guasti pre-schedulati, nessuna scorciatoia: il guasto iniettato dal vivo attraversa il topic `telemetry` reale.
- **Robot di riserva dedicati** (R4, e R8 nello scenario esteso): fermi finché non servono, mai scelti fra i robot già in missione — garantisce un sostituto sempre pronto.
- **Anello automatico**: quando un'anomalia di salute su *soglia fissa* (deterministica) viene rilevata su un robot reale, il backend invia il robot colpito verso un nodo di riparazione e disaccia una riserva disponibile sulla missione originale (letta da `config/experiment.json`) — ripartendo da capo, non dal punto esatto di interruzione (limite dichiarato, §4.2).

Nota tecnica di rilievo — falsi positivi statistici collegati a un'azione reale. La prima versione dell'anello automatico si agganciava al flag `health_anomaly` di `fleet_state` , che è l'OR fra soglie fisse e Isolation Forest. Con 8 robot attivi (scenario esteso), il tasso di falsi positivi *strutturale* dell'Isolation Forest (parametro `contamination=0.02` : per definizione, circa il 2% delle predizioni scatta sempre, anche su dati perfettamente nominali) è diventato consequenziale — quasi tutta la flotta finiva "in riparazione" entro un minuto senza alcun guasto iniettato. La correzione risolutiva non è stata continuare a tarare il modello statistico, ma **cambiare quale segnale aziona l'azione reale**: l'anello automatico ora ascolta solo le anomalie di salute su soglia fissa (deterministiche per costruzione), lasciando l'Isolation Forest attivo altrove nel sistema per il suo valore di segnale "morbido". È una lezione generale, non specifica di questo progetto: un modello statistico con un tasso di falsi positivi noto per costruzione può essere accettabile come segnale informativo passivo, ma richiede cautela esplicita quando viene collegato per la prima volta a un'azione reale.

Passo 15: previsione live, reazione differenziata, deposito dedicato

Usando il sistema dopo il Passo 14, sono emerse due lacune concrete: la previsione dei guasti (§2.8) restava uno script offline, mentre l'anello di self healing mandava *ogni* anomalia di salute confermata in riparazione automatica — anche un guasto già persistente, per cui una manovra preventiva non ha più senso. L'estensione introduce una reazione *differenziata* in base a quanto il problema è già maturo:

- **Nuovo guasto preavviso intermittente** : a differenza delle firme esistenti (rampa continua o picco sostenuto), tiene un canale oltre una soglia "morbida" solo a raffiche saltuarie (pochi secondi ogni `burst_interval_s`), tornando nominale in mezzo — simula un robot che si sta avvicinando a un guasto, non uno già rotto.
- **Quarto operatore stateful in `detection_job.py`** , dettagli tecnici in §2.6: emette un'anomalia `type="previsione"` calcolata interamente in streaming, non in un batch offline separato — il primo segnale di rischio (§2.6, tabella "rischio/anomalia/danno") a essere prodotto anche dalla speed layer, non solo dal batch layer (§2.8).
- **Reazione differenziata**: una *previsione* (il guasto non è ancora confermato) attiva la stessa riparazione preventiva + dispaccio riserva del Passo 14; un guasto *persistente confermato* (soglia fissa) non manda più il robot in riparazione automatica — lo ferma dove si trova (nuovo comando `-nav_control {"cmd":"freeze"}`) e attende che un operatore lo osservi in dashboard e lo rimuova esplicitamente dalla flotta (nuovo stato "in avaria", bottone "Decommissiona"). Motivazione: un guasto già confermato è per definizione troppo tardi per una manovra preventiva.
- **Deposito dedicato**: verificando il ciclo di riparazione è emersa una collisione fisica reale — `repair_node` e `reserve_node` coincidevano con lo `start_node` di robot reali ancora in servizio (R1, R4). Tre nodi nuovi fuori dai corridoi principali (`DEPOSITO` , `RISERVA1` , `RISERVA2`), ciascuno con un raccordo dedicato da un nodo esistente, eliminano la collisione — un'eccezione registrata esplicitamente alla scelta "nessun cambio di topologia" del Passo 14.
- **Isolamento fra run (`run_id`)**: i robot-token del generatore sintetico riusano lo stesso `robot_id` a ogni run (es. `SIM00000`); senza un identificatore di sessione, l'analisi storica (previsione offline, query del layer TAG) mischierebbe run diversi. Un `run_id` nullable, generato una volta per run e propagato end-to-end (telemetria, anomalie, guasti iniettati, previsioni), isola l'analisi senza mai cancellare dati grezzi — coerente con l'idea che in un sistema Big Data lo storico non si cancella, si filtra. **Colonna normale, non chiave di partizione** (nota tecnica sotto): un primo tentativo di partizionare anche per `run_id` ha rotto la lettura dell'intero storico preesistente.

Tre bug reali trovati verificando dal vivo, non solo nel codice nuovo. (1) Cambiare lo schema di stato di un operatore `applyInPandasWithState` senza cancellare i checkpoint Spark (`/tmp/shf-checkpoints`) causa un crash-loop silenzioso (`InvalidUnsafeRowException` , mascherato da un generico `py4j.Py4JException: Received empty command`) — non specifico a questo progetto, capita a chiunque cambi lo stato di un operatore stateful senza pulire i checkpoint. (2) Un robot "decommissionato" dall'operatore continuava a ripubblicare `fleet_state` (il nodo ROS non sa nulla della rimozione lato dashboard), rimettendolo nella Map in memoria del backend al messaggio successivo — il consumer ora scarta esplicitamente i messaggi di un robot già decommissionato. (3) `spark.read.parquet` senza l'opzione `mergeSchema` fallisce (non solo mischia i dati) su una query che referencia una colonna nuova (`run_id`) se anche un solo file più vecchio nella cartella non ce l'ha ancora — Parquet è colonnare per file fisico, un campo nullable nello schema logico non basta. (4) Un primo tentativo di rendere `run_id` anche una chiave di **partizione** (oltre a `type` per `anomalies`) ha rotto la lettura dell'intera tabella `anomalies` , non solo delle righe vecchie: i dati preesistenti erano scritti a una profondità di partizione diversa (`type=X/*.parquet` piatto) da quelli nuovi (`type=X/run_id=Y/*.parquet`), e Spark pretende uno schema di partizione uniforme su tutta la directory (`AssertionError: Conflicting partition column names detected`). Anche dopo aver ripulito i file, l'errore persisteva perché `_spark_metadata` (il log dei batch di Structured Streaming) ricordava ancora la struttura incoerente — è stato necessario cancellare anche quello. Decisione finale: `run_id` resta una colonna normale ovunque, mai una chiave di partizione.

Passo 16: perturbazioni reattive, streaming live in dashboard, fix riserve/deposito

Dopo altro uso reale del sistema sono emerse due richieste di interattività sulla GUI e tre bug, tutti verificati nel codice prima di intervenire:

- **Perturbazioni reattive (`rumore_sensore`)**: distinto deliberatamente da un "guasto" — rumore gaussiano extra su un canale (non una firma strutturata), iniettabile dal vivo con la stessa via reattiva dei guasti esistenti, ma **esclusa dal ground truth** `injected_faults` : se vi finisce, il job di calibrazione delle soglie adattive (§2.7) la conterebbe come guasto vero invece che come il falso positivo che deve imparare a filtrare. Serve a generare falsi positivi controllati per verificare che il sistema li impari a filtrare.

- **Streaming live dei topic Kafka:** nuovo pannello dashboard che mostra i messaggi grezzi campionati (al più uno ogni ~200ms per topic) di `telemetry / anomalies / injected_faults / fleet_state`, via un consumer Kafka indipendente e lo stesso canale WebSocket già in uso — nessuna persistenza, buffer solo lato client.
- **Fix: riserve che non ricomparivano:** lo stato in memoria del backend (riserve già disacciate, robot in riparazione, robot decommissionati) viveva per l'intera vita del processo, non per singola simulazione — un nuovo avvio della flotta reale non lo azzerava, quindi una riserva usata in un run restava inutilizzabile per sempre nei run successivi. Corretto con un reset esplicito a ogni avvio.
- **Fix: robot che finivano al deposito senza un guasto:** il generatore sintetico trattava `DEPOSITO / RISERVA1 / RISERVA2` (nodi dedicati alla flotta reale, Passo 15) come nodi qualunque del grafo, raggiungibili per un normale random walk. Esclusi esplicitamente dallo spawn iniziale e dall'adiacenza percorribile.
- **Fix: pannello "Robot a rischio":** era bloccato su un dato di test residuo di una sessione precedente (rimosso) e mostrava solo le previsioni offline batch, cieco alle previsioni `type="previsione"` già visibili in tempo reale in "Eventi recenti". Ora unisce le due fonti, con precedenza a quella live (più fresca) e una scadenza di 2 minuti (il suo `lead_time_s` è una stima fissa, non decade da sola come una regressione).

Bug reali trovati verificando dal vivo. (1) `docker restart` non basta per il codice del backend Node né per quello del generatore sintetico: entrambi copiano il codice nell'immagine Docker al build, non lo montano live — serve `docker compose build` prima di ricreare il container. Il generatore sintetico, in più, gira come processo Python persistente già importato in memoria (`supervisord`): anche con un bind mount live del codice sorgente, serve un riavvio esplicito del processo (`supervisorctl restart`) perché il modulo venga ricaricato. (2) Il checkpoint di `persistence_job.py` e il log di sink `_spark_metadata` (scritto dentro ciascuna cartella dati) possono disallinearsi se uno dei due viene resettato senza l'altro (già capitato al Passo 15 per un problema di partizionamento): il job pensa di aver già scritto batch mai arrivati su disco, e fallisce o — peggio — "committa" batch senza scriverli davvero, in silenzio. Fix: cancellarli sempre insieme, mai uno solo; aggiunta anche un'opzione per limitare quanti messaggi legge per trigger, così un recupero di un backlog enorme (milioni di messaggi) progredisce a piccoli passi osservabili invece di restare bloccato in un unico batch enorme. (3) **Kafka non ha un volume Docker persistente** in questo progetto: un riavvio dei container (qui capitato per un blocco del PC usato per lo sviluppo) fa ripartire Kafka completamente vuoto, azzerando tutti i topic, mentre lo storico Parquet e i checkpoint (su un volume persistente separato) sopravvivono — un'asimmetria che, se non gestita, fa fallire il job di persistenza al riavvio. Non risolto in questa sessione (richiederebbe un volume dedicato per Kafka, con conseguenze di spazio disco da valutare), solo documentato come limite noto (§4.2).

2.12 Struttura del database (storico Parquet)

Perché Parquet, e cosa cambierebbe con le alternative

Parquet è un formato **colonnare**: a differenza di CSV o JSON Lines, che scrivono una riga intera per record, Parquet raggruppa i valori per colonna dentro blocchi compressi (*row group*), con lo schema incorporato nel file stesso. Per il pattern di lettura di questo progetto — il layer TAG, gli script `eval/` e `forecast_failures.py` quasi sempre selezionano poche colonne (es. solo `motor_temp` di un robot) su uno storico che cresce nel tempo — questo conta doppiamente: Spark legge da disco solo le colonne richieste (*column pruning*), e la compressione per colonna (valori simili vicini) è più efficace di quella per riga. Un CSV o un JSON Lines equivalente andrebbe letto per intero anche per una sola colonna, ed è più grande su disco a parità di dati (nessuna compressione colonnare, chiavi ripetute ad ogni riga nel caso JSON).

Cosa cambierebbe con altri formati candidati:

- **CSV:** nessun tipo nativo (tutto stringa finché non riparsato), nessuna struttura annidata — `threshold_reasons` (array) e `params` di `injected_faults` (oggetto nidificato per tipo di guasto) andrebbero serializzati a mano in una stringa, perdendo la possibilità di interrogarli direttamente via SQL.
- **JSON Lines:** mantiene la struttura annidata (bene), è lo stesso formato già usato sul wire di Kafka (§2.4) — ma resta un formato a righe: va deserializzata l'intera riga anche per leggere una sola colonna, e senza compressione colonnare il footprint su disco cresce più in fretta.
- **Avro:** binario, schema incorporato, ottimo supporto alla evoluzione dello schema — probabilmente la scelta più naturale *sul wire* di uno stream (non è quella adottata qui, §2.2 spiega perché si è preferito JSON anche lì), ma resta un formato a righe: stesso svantaggio di CSV/JSON per query analitiche che leggono poche colonne su molte righe.
- **ORC:** colonnare come Parquet, prestazioni comparabili, storicamente più legato all'ecosistema Hive. Parquet resta il default più diffuso nell'ecosistema Spark, con supporto nativo senza configurazione aggiuntiva — la scelta riflette questo, non una misurazione che lo dichiara superiore a ORC per questo caso d'uso.
- **Delta Lake** (Parquet + log delle transazioni: ACID, time travel, upsert sicuri in concorrenza): valutato in fase di pianificazione iniziale, poi semplificato al solo Parquet. Lo storico di questo progetto è *append-only* per costruzione (un solo scrittore per tabella, mai un update o una cancellazione di righe esistenti — §2.15), quindi il vantaggio principale di Delta non si materializza; il costo concreto (una dipendenza Maven in più da risolvere ad ogni avvio, un log delle transazioni da mantenere) non era giustificato a questo volume di dati.

Un effetto collaterale del formato, scoperto durante lo sviluppo: le colonne di partizione (es. `type` per `anomalies`, §2.7) **non** vengono scritte dentro ai singoli file Parquet — vivono solo nel path della cartella (`type=salute/...`), ricostruite dal lettore. Un primo tentativo di `offline/adaptive_thresholds.py` di leggere i singoli file Parquet uno a uno (invece della cartella intera) falliva con un `KeyError` proprio sulla colonna di partizione — un dettaglio non ovvio, specifico di come Parquet/Spark gestiscono il partizionamento in stile Hive.

Non esiste un RDBMS tradizionale: lo "storico" è un insieme di quattro cartelle Parquet su un volume Docker condiviso (`shf-data`, montato su `/data`), scritte dal batch layer (§2.7, §2.8) e lette come tabelle SQL da `query_service.py` (§2.9), che ne registra una vista temporanea per ciascuna (`createOrReplaceTempView`) — sono esattamente le quattro tabelle descritte nello schema del prompt del layer TAG. Gli schemi Spark sono centralizzati in `streaming/schemas.py`, condivisi fra il job di detection e quello di persistenza.

telemetry

Un record per ogni tick di ogni robot (reale o sintetico), **non partizionata**. Stesso schema del messaggio Kafka (§2.3).

Colonna	Tipo	Descrizione
ts	long	epoch millisecondi
robot_id	string	es. R1 (reale) o SIM00042 (sintetico)
run_id	string (nullable)	identifica la sessione/run corrente (Passo 15); null per dati storici precedenti al campo
x, y, theta	double	posa sul piano
v_lin, v_ang	double	velocità reali (odometria)
cmd_v_lin, cmd_v_ang	double	velocità comandate
battery_pct, motor_current, motor_temp	double	canali di salute (sintetizzati, §2.2)
min_obstacle_dist	double	lidar, metri
task_state	string	idle / moving / blocked / charging
current_edge, goal_node	string	posizione sul grafo del magazzino

anomalies

Partizionata per type (salute / livelock / deadlock / previsione , sottocartelle type=... /) — le query a valle filtrano quasi sempre per tipo, il partitioning evita di scansionare l'intero dataset. Lo schema è l'unione dei campi emessi dai quattro meccanismi di detection (§2.6): per un dato type , i campi non pertinenti restano null . previsione (Passo 15) non è un guasto rilevato ma un preavviso calcolato in streaming — riusa i nomi di campo delle previsioni offline (channel , current_value , critical_threshold , lead_time_s , non in tabella perché condivisi con predictions sotto) per coerenza.

Colonna	Tipo	Pertinente a	Descrizione
type	string	tutti	chiave di partizione
ts	long	tutti	epoch millisecondi dell'evento
run_id	string (nullable)	tutti	isolamento fra run (Passo 15) — colonna normale, filtrata via WHERE , non una chiave di partizione (vedi nota sotto)
robot_id	string	salute, livelock, previsione	assente per deadlock (coinvolge più robot, vedi robots)
threshold_reasons	array<string>	salute	quali soglie fisse hanno scattato (es. ["motor_current"])
if_anomaly	int (0/1)	salute	esito dell'Isolation Forest
motor_temp, motor_current, battery_pct	double	salute	valori dei canali al momento dell'anomalia
window_start, window_end	string	livelock, deadlock	estremi della finestra/checkpoint temporale
min_dist, max_dist, stall_duration_s	double	livelock	escursione della distanza dal goal e durata dello stallo confermato
n_msgs, n_moving	long	livelock, deadlock	conteggi diagnostici (quanti messaggi/robot in moto nella finestra)
current_edge	string	deadlock	arco a corsia singola conteso
robots	array<string>	deadlock	i robot coinvolti nello stallo

injected_faults

Non partizionata — è la ground truth sperimentale (§3.1). Un record per ogni guasto, scritto alla sua *disattivazione* (quando si conoscono già entrambi gli estremi reali).

Colonna	Tipo	Descrizione
fault_id	string	identificativo univoco (es. F1 , o LIVE-a3f9c21b per un'iniezione dal vivo)
run_id	string (nullable)	isolamento fra run (Passo 15) — colonna normale, non una chiave di partizione
robot_id, fault_type	string	chi e quale delle 5 firme di salute (§2.2, §2.11)
start_time_s, end_time_s	double	orario pianificato, relativo all'inizio della run
params	struct (nested)	parametri della firma — superset di tutti i tipi, i non pertinenti sono null (es. peak_a per spike_corrente , plateau_temp_c per deriva_termica)
start_ts, end_ts	long	orario reale (epoch ms) di attivazione/disattivazione — il campo usato per il confronto con le anomalie rilevate (§3.1)

predictions

Non partizionata, un file per ogni run di `forecast_failures.py` (§2.8) — a differenza delle altre tre, non è scritta da una query Structured Streaming ma da uno script batch invocato periodicamente.

Colonna	Tipo	Descrizione
<code>robot_id</code> , <code>channel</code>	string	quale robot, quale canale di salute
<code>run_id</code>	string (nullable)	isolamento fra run (Passo 15): di default lo script analizza solo l'ultimo <code>run_id</code> presente nello storico
<code>predicted_at_ts</code>	long	istante (epoch ms) in cui la previsione è stata calcolata
<code>current_value</code> , <code>slope_per_min</code>	double	ultimo valore osservato e pendenza stimata
<code>critical_threshold</code>	double	soglia critica del canale (§2.8)
<code>predicted_crossing_ts</code>	long	istante previsto di superamento soglia
<code>lead_time_s</code>	double	<code>predicted_crossing_ts - predicted_at_ts</code> , in secondi
<code>model</code>	string	sempre "regressione lineare (OLS)" nella versione attuale
<code>n_points</code>	int	quanti punti ricampionati sono entrati nel fit

2.13 Il percorso dei dati: dalla sorgente alla visualizzazione (e viceversa)

Due percorsi distinti attraversano il sistema: uno porta la **telemetria** dal robot fino alla dashboard (e allo storico), l'altro porta un **comando** dalla dashboard fino al robot. Il secondo, quando esiste (guasto iniettato dal vivo, avvio/arresto della simulazione, missione assegnata), si richiude sempre nel primo: un comando accettato si manifesta come telemetria reale che rifà tutto il giro del percorso A.

Percorso A — dalla telemetria alla dashboard

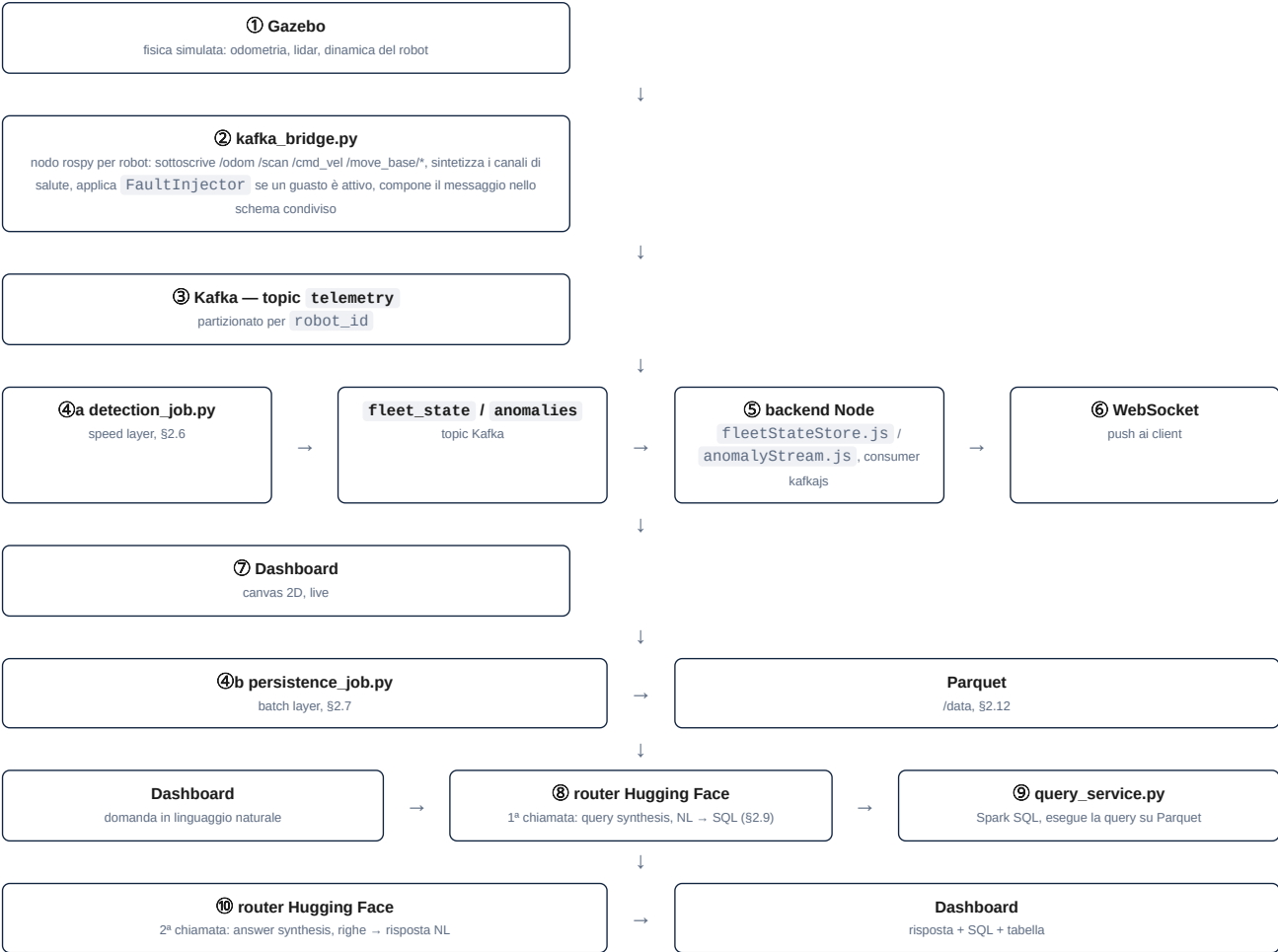


Fig. 2 — Percorso A: dalla fisica simulata alla dashboard, per la via rapida (④a, real-time) e per quella storica (④b, batch → query ad-hoc). `forecast_failures.py` e `adaptive_thresholds.py` (§2.7, §2.8) si inseriscono anch'essi qui, leggendo dallo stesso Parquet. Il router Hugging Face compare due volte (⑧, ⑩): stessa infrastruttura, due chiamate con prompt diversi — è la struttura a tre stadi del layer TAG (§2.9).

Percorso B — dalla dashboard al robot (comandi)



Fig. 3 — Percorso B: un'azione dell'utente non modifica mai lo stato del robot "a distanza" — arriva come comando su un topic ROS o come richiesta a `supervisorctl`, ed è il nodo ROS stesso ad agire, generando poi telemetria reale che il sistema di detection vede come vedrebbe qualunque altro evento.

Sotto i due percorsi: i connettori usati per ogni hop verso/da Kafka

I riquadri di Fig. 2/3 nascondono meccanismi di connessione diversi a seconda del linguaggio e del ruolo di chi parla con Kafka:

- **ROS → Kafka.** `kafka_bridge.py` usa `confluent_kafka.Producer`, un client Python che apre una connessione diretta al broker (`kafka_bootstrap`, rosparam risolto via DNS Docker come `kafka:9092`) e pubblica in modo asincrono, fire-and-forget (`producer.poll(0)`, non bloccante): un errore di consegna viene solo loggato, non blocca il tick di telemetria successivo.
- **Kafka → Spark.** Sia `detection_job.py` sia `persistence_job.py` leggono/scrivono Kafka tramite il connettore ufficiale `spark-sql-kafka-0-10_2.12` (pacchetto Maven risolto a runtime da `spark-submit --packages`, non incluso nell'immagine base): un topic è visto come un normale `readStream`/sink Structured Streaming, non una libreria client da gestire a mano come lato ROS.
- **Kafka → backend Node.** Tre consumer `kafka.js` indipendenti (`fleetStateStore.js`, `anomalyStream.js`, `rawStream.js`), ciascuno con un proprio group id: il backend non fa da semplice inoltro dei messaggi grezzi, li trasforma (aggrega lo stato flotta, decide le reazioni di self healing) prima di ributarli sul WebSocket verso il browser — che non parla mai con Kafka direttamente.
- **Chi non parla affatto con Kafka.** `query_service.py` (§2.9) legge solo Parquet, mai un topic: coerente con l'essere un servizio di interrogazione ad-hoc sullo storico, non un consumatore di flusso.

Un dettaglio operativo non visibile nei diagrammi: `detection_job.py` e `query_service.py` partono da soli con lo stack (`start-master.sh`, §2.14), mentre `persistence_job.py` va lanciato manualmente (`docker exec`, stesso comando `spark-submit --packages ...` visto sopra) — senza, lo storico Parquet non cresce, e gli esperimenti che ne dipendono (execution accuracy del TAG, §3.1) non hanno dati recenti su cui lavorare.

2.14 Orchestrazione e deployment

L'intero sistema è orchestrato con Docker Compose: cinque container — `kafka`, due Spark (`spark-master` / `spark-worker`), `backend` (Node, serve anche la dashboard statica), `ros` (basato su `osrf/ros:noetic-desktop-full` con i pacchetti TurtleBot3, noVNC per l'accesso GUI in debug). L'intero stack si avvia con un singolo `docker compose up -d`, senza comandi manuali post-avvio — con una sola eccezione dichiarata più sotto (`persistence_job.py`).

Chi lancia cosa: non tutti gli script partono allo stesso modo

Non c'è un meccanismo di avvio unico: ogni script/processo usa il meccanismo più adatto al proprio ruolo, non per incoerenza ma perché "parte da solo sempre", "parte da solo ma è disattivabile" e "va lanciato quando serve" sono tre esigenze diverse.

Script/processo	Container	Come parte	Individuale o gruppo?
Kafka broker	kafka	entrypoint dell'immagine <code>apache/kafka</code> , nessuno script custom	—
<code>query_service.py</code>	<code>spark-master</code>	<code>start-master.sh</code> (override di <code>command:</code> in <code>docker-compose.yml</code>), loop con retry automatico	Automatico, con <code>detection_job.py</code>
<code>detection_job.py</code>	<code>spark-master</code>	stesso <code>start-master.sh</code> , secondo <code>spark-submit</code> nello stesso script	Automatico, con <code>query_service.py</code>
<code>persistence_job.py</code>	<code>spark-master / worker</code>	manuale: <code>docker exec ... spark-submit ... persistence_job.py</code> — nessuno script lo avvia da solo	Individuale, a comando
Spark worker	<code>spark-worker</code>	CMD originale dell'immagine bitnami, si registra da solo al master	Automatico
backend Node + dashboard	<code>backend</code>	<code>npm start</code> (CMD del Dockerfile), un solo processo	Automatico
<code>roscore</code> , <code>vncserver</code> , <code>novnc</code>	<code>ros</code>	programmi <code>supervisord</code> , <code>autostart=true</code>	Automatico, indipendenti
<code>generator_service.py</code>	<code>ros</code>	programma <code>supervisord</code> , <code>autostart=true</code> — resta in ascolto HTTP, il generatore vero e proprio parte solo su richiesta (<code>POST /start</code>)	Automatico (il servizio), on-demand (il run)
<code>fleet_control_service.py</code>	<code>ros</code>	programma <code>supervisord</code> , <code>autostart=true</code> (con 5s di ritardo: aspetta <code>roscore</code>)	Automatico
<code>eval_service.py</code>	<code>ros</code>	programma <code>supervisord</code> , <code>autostart=true</code> — in ascolto, i run partono su <code>POST /run</code>	Automatico (il servizio), on-demand (il run)
<code>sim_multi_robot</code> (Gazebo + tutti i nodi robot)	<code>ros</code>	programma <code>supervisord</code> , <code>autostart=false</code> : <code>fleet_control_service.py</code> lo avvia/ferma con <code>supervisorctl start/stop</code> su richiesta della dashboard	Gruppo (un <code>roslaunch</code> avvia tutti i nodi della flotta insieme)

Il caso di `persistence_job.py` merita una nota esplicita, già citata in §2.13: è l'unico componente della pipeline "core" senza autostart. Una scelta pragmatica più che architetturale — durante lo sviluppo va fermato/riavviato spesso per ripulire checkpoint (§2.7), e automatizzarlo avrebbe richiesto lo stesso meccanismo di retry-loop di `start-master.sh` senza un beneficio operativo chiaro finché non lo si tratta come un servizio da "mettere e dimenticare" — una candidatura naturale per gli sviluppi futuri (§4.3).

Dentro il container `ros`, `sim_multi_robot` non è un solo processo ma un `roslaunch` che avvia in un colpo solo `roscore` (già su, separato), Gazebo, e per ciascun robot della flotta (scala `small` o `large`) una terna `move_base` + `graph_navigator.py` + `kafka_bridge.py` nel proprio namespace — coerente con l'essere "un gruppo", non componenti indipendenti: non ha senso far partire la navigazione di un robot senza la sua simulazione fisica.

Rete: come i container si trovano

Nessuna configurazione di rete esplicita in `docker-compose.yml`: Compose crea automaticamente una rete bridge dedicata al progetto, e ogni servizio vi si connette con il proprio nome come hostname DNS interno — `kafka:9092`, `spark-master:7077`, `backend:3000` sono risoluzioni DNS Docker, non indirizzi IP fissi né configurazione manuale. Le variabili d'ambiente dei vari servizi (`KAFKA_BOOTSTRAP`, `QUERY_SERVICE_URL`, `BACKEND_URL`, ecc., §2.13) contengono questi nomi, non IP. Solo le porte elencate in `ports` sono raggiungibili anche dall'host (es. `3000` per la dashboard, `9094` per Kafka da un client esterno, `6080` per noVNC) — la comunicazione fra container passa sempre per la rete interna, mai attraverso l'host.

2.15 Pulizia e validazione dei dati lungo la pipeline

Non c'è un unico "step di pulizia" centralizzato: ogni stadio della pipeline valida/pulisce solo ciò che gli serve per non propagare un problema a valle, con la tecnica più adatta a quello stadio.

Sia in streaming sia fuori, ma con tecniche diverse per un motivo preciso. Il lato in streaming (ingresso ROS/Kafka, §2.2, e `detection_job.py`, §2.6) può permettersi solo controlli *a costo pressoché nullo per messaggio* — un confronto `is not None`, un `fillna` su una manciata di colonne — perché ogni micro-batch ha un budget di tempo stretto (trigger di 2-10s, §2.6): un controllo costoso ripetuto ad ogni riga farebbe accumulare ritardo. Il lato non in streaming (`forecast_failures.py`, `offline/adaptive_thresholds.py`, §2.7-§2.8, eseguiti on-demand o periodicamente, non su un trigger stretto) può permettersi controlli più costosi sull'intero dataset in una volta — scansionare una cartella intera cercando file da 0 byte, o un `dropna()` su tutta la colonna prima di un fit — proprio perché non ha lo stesso vincolo di tempo reale. Sono due filosofie diverse per lo stesso obiettivo, non un'incoerenza: pulizia a basso costo e localizzata dove il tempo è poco, pulizia più esaustiva dove il tempo c'è.

- **All'ingresso (ROS → Kafka), §2.2:** `/scan` arriva con letture `LaserScan.ranges[]` che possono includere valori fuori dal range fisico del sensore (troppo vicino/lontano, letture spurie) — `kafka_bridge.py` filtra esplicitamente per `range_min / range_max` prima di calcolare `min_obstacle_dist`, scartando le letture non valide invece di lasciarle inquinare il minimo.
- **Messaggi malformati, ovunque venga fatto parsing:** sia i consumer Kafka del backend (`fleetStateStore.js`, `anomalyStream.js`) sia gli script `eval / test /` avvolgono ogni `JSON.parse / json.loads` in un blocco che scarta silenziosamente il singolo messaggio non deserializzabile invece di far cadere l'intero consumer — un messaggio corrotto raro non deve fermare il flusso di tutti gli altri. Lato Spark, `from_json` con uno schema esplicito (§2.3, `schemas.py`) ha lo stesso effetto per costruzione: un campo mancante o malformato diventa `null` nella riga, non un'eccezione che interrompe il micro-batch.
- **Nello streaming di detection (§2.6):** i controlli soglia (`threshold_reasons_udf`) verificano esplicitamente che ciascun valore non sia `None` prima di confrontarlo con una soglia — un robot appena connesso, senza ancora una lettura valida per un canale, non deve generare un'anomalia falsa né far fallire lo Spark job. L'Isolation Forest, che non tollera `NaN` in input, applica `fillna({"v_lin": 0.0, "min_obstacle_dist": 3.5})` prima della predizione — valori

di default scelti come "neutri" (fermo, area aperta), non zero ovunque: un default arbitrario avrebbe potuto far scattare un'anomalia spuria su un semplice dato mancante.

- **Nello storico Parquet (§2.7, §2.8):** `forecast_failures.py` rimuove esplicitamente eventuali file `.parquet` da 0 byte prima di leggere una cartella (capita se lo streaming writer viene interrotto a metà scrittura — un file vuoto manda in crash `pandas.read_parquet` se letto direttamente, bypassando il commit log `_spark_metadata` di Spark che lo escluderebbe). Sia `forecast_failures.py` sia `offline/adaptive_thresholds.py` applicano `dropna()` sulla colonna di interesse prima di un fit di regressione o del calcolo di un percentile — un `NaN` silenzioso in ingresso a `numpy.polyfit` produrrebbe un risultato spazzatura senza sollevare errore.
- **In uscita, verso i consumatori JSON (backend, dashboard, eval/):** `NaN/Infinity` sono float Python legali ma non JSON valido (§4.1) — `json_safe()` in `eval/common.py` li converte ricorsivamente in `null` prima di ogni scrittura di `index.json` o risposta HTTP, altrimenti un valore statisticamente legittimo (es. precisione con `TP+FP=0`) romperebbe silenziosamente il parsing lato client.

Un limite dichiarato, non nascosto: non esiste una fase di *deduplicazione* esplicita in nessun punto della pipeline. Rileva solo in caso di rilettura da un checkpoint perso (§2.7) — scenario raro, documentato come limite noto in §4.2, non gestito con una logica ad hoc (es. una chiave di idempotenza) perché il volume/la frequenza con cui accade in pratica non l'hanno reso finora necessario.

2.16 La suite di verifica (test/): cosa controlla, come

Distinta da `eval/` (§3): qui si verifica **correttezza** (pass/fail), non si producono numeri per la tesina — stesso principio della differenza già discussa fra "guasto" e "previsione" (§2.9), applicato questa volta alla metodologia sperimentale invece che al dominio applicativo. Gira dentro il container `ros` (ha già `confluent-kafka / pandas / pyarrow` e vede tutti i servizi sulla rete Docker interna).

`test/conftest.py` — fixture condivise, stesso stile di `eval/common.py` (client Kafka, helper HTTP verso `query_service.py / backend/generatore`) più due fixture `autouse` specifiche di questa suite:

- `_pause_real_simulation` (session-scoped): mette in pausa `sim_multi_robot` via `supervisorctl` per tutta la sessione di test (Gazebo è pesante in CPU e non serve — i test usano il generatore sintetico per avere robot controllati), e la rimette su alla fine *solo se* era già in esecuzione prima — un controllo esplicito, perché `supervisorctl stop` ritorna comunque successo anche se il programma era già fermo.
- `_ensure_generator_idle` (per-test): ferma il generatore prima e dopo ogni test, così' un test fallito a metà non lascia un run appeso che fa fallire i successivi con `409`.

Sei file, organizzati per area della pipeline, non per tecnologia:

File	Cosa verifica
<code>test_schema.py</code>	i messaggi prodotti rispettano lo schema condiviso (§2.3) — telemetria, <code>fleet_state</code> , <code>injected_faults</code> , <code>predictions</code>
<code>test_detection_effectiveness.py</code>	veri/falsi positivi puntuali per salute, deadlock, livelock (scenari mirati, non uno sweep statistico come in <code>eval/</code>)
<code>test_fault_ground_truth.py</code>	un guasto iniettato produce un record <code>injected_faults</code> coerente (timestamp, robot corretto anche con <code>robot_id="random"</code>)
<code>test_prediction_accuracy.py</code>	la previsione offline (§2.8) converge vicino al valore analitico atteso, e <i>non</i> scatta su un canale stabile o su un trend nella direzione sbagliata (test di non-falso-positivo, non solo di accuratezza)
<code>test_tag_accuracy.py</code>	risposte del layer TAG confrontate con una query di verità diretta, più la guardia anti- <code>SELECT -only</code>
<code>test_efficiency.py</code>	soglie minime di throughput/latenza (pass/fail, non i numeri completi dello sweep di §3.2)

3. Risultati

3.1 Effectiveness: precisione della detection, della previsione e del TAG

Il meccanismo di ground truth

Nessuna delle metriche seguenti è stimata "a occhio": tutte si appoggiano a un meccanismo comune, oggettivo e riproducibile. Ogni volta che un guasto viene iniettato — dal vivo o pre-schedulato, sulla flotta reale o sul generatore sintetico (§2.2, §2.5) — un record viene scritto sul topic `injected_faults` con `robot_id`, `fault_type`, e l'intervallo `[start_ts, end_ts]` in cui il guasto è stato effettivamente attivo (timestamp reali di attivazione/disattivazione, non quelli pianificati). Questo record è la **ground truth**: una verità nota a priori, indipendente da qualunque cosa il sistema di detection creda di aver visto.

Il confronto è sempre lo stesso, sia negli esperimenti di `eval/` sia nel job offline di calibrazione delle soglie (`offline/adaptive_thresholds.py`, §2.7):

- un'anomalia rilevata (evento su `anomalies`) è un **vero positivo** se il suo timestamp cade dentro la finestra `[start_ts, end_ts]` di un guasto realmente attivo per quello stesso `robot_id`;
- un'anomalia rilevata **fuori** da qualunque finestra di guasto attivo per quel robot è un **falso positivo** — il sistema ha segnalato qualcosa che, per la ground truth, non stava succedendo;
- un guasto la cui finestra non produce **nessuna** anomalia corrispondente è un **falso negativo** — il sistema non si è accorto di un guasto realmente avvenuto;
- un robot senza guasti attivi e senza anomalie rilevate è un **vero negativo**.

Questo è possibile solo perché l'iniezione dei guasti passa dallo stesso percorso dati della telemetria reale (§2.2): la ground truth e il segnale osservato dal detector condividono lo stesso orologio e lo stesso `robot_id`, quindi il confronto non richiede alcuna correlazione manuale o approssimata.

Detection di salute — come si conta vero/falso positivo in pratica

L'esperimento (`eval/run_effectiveness.py`, funzione `run_detection_experiment`) mette in campo 8 robot-token del generatore sintetico; a 3 di essi (scelti in anticipo, indici noti) viene iniettato un guasto `spike_corrente` di 15 secondi; i restanti 5 non ricevono alcun guasto. Uno script si mette in ascolto sul topic `anomalies` per l'intera durata del run (più un margine) e registra **quali robot_id compaiono almeno una volta** come evento `type="salute"`. A questo punto, per ciascuno degli 8 robot si applica meccanicamente la tabella di verità:

	rilevato (compare su <code>anomalies</code>)	non rilevato
guasto realmente iniettato	vero positivo (TP)	falso negativo (FN)
nessun guasto iniettato	falso positivo (FP)	vero negativo (TN)

Da qui, le formule standard: $\text{precision} = \text{TP} / (\text{TP} + \text{FP})$ (fra tutti gli allarmi lanciati, quanti erano guasti veri), $\text{recall} = \text{TP} / (\text{TP} + \text{FN})$ (fra tutti i guasti veri, quanti sono stati notati), $F1$ la loro media armonica. Nel run più recente: TP=3 (tutti e 3 i robot guasti rilevati), FP=3, FN=0, TN=2 — precision=0.50, recall=1.00, F1=0.67.

Un dettaglio importante per l'interpretazione: questa metrica è *binaria per robot* ("è mai stato rilevato durante il guasto, sì/no"), non conta quante volte l'anomalia è stata ri-emessa nella finestra (il job di detection, §2.6, scrive un evento ad ogni trigger di 2s in cui la soglia/il modello restano sopra soglia — durante 15s di guasto attivo, il conteggio grezzo di eventi sarebbe quindi tipicamente 6-8, non 1). Contare "rilevato/non rilevato" per robot, invece che il numero grezzo di eventi, evita di gonfiare artificialmente precision/recall sommando eventi che sono in realtà la stessa anomalia osservata più volte.

Il recall è stabile fra run diversi, la precision no — un'osservazione empirica, non solo teorica: lo stesso esperimento, eseguito in tre occasioni diverse nel corso dello sviluppo, ha prodotto (precision, recall): (1.00, 1.00), (indefinita, 0.00 — nessun vero positivo rilevato) e (0.50, 1.00). Il recall vicino a 1.00 nella maggioranza dei run conferma che il segnale di guasto vero (`spike_corrente`) è quasi sempre abbastanza forte da superare la soglia fissa quando è davvero attivo; la precision oscilla perché con solo 8 robot e una finestra di osservazione di 25s bastano 1-2 falsi positivi in più o in meno — dovuti al rumore del random walk del generatore che occasionalmente sconfina oltre soglia anche su un robot "sano" — per spostare il rapporto TP/(TP+FP) in modo vistoso. È lo stesso motivo, alla radice, del collo di bottiglia di precision osservato a scala molto più grande in §3.3 (Fig. 5): con soglie fisse e non ricalibrate su questi run brevi, il tasso di falsi positivi per singolo robot resta piccolo ma non nullo, e diventa evidente sia quando i robot totali sono pochi (varianza statistica) sia quando sono moltissimi (accumulo).

Previsione in streaming (Passo 15/16) — lo stesso esperimento, sul quarto operatore

Lo stesso schema di ground truth si applica al segnale `type="previsione"` del quarto operatore stateful di `detection_job.py` (§2.6): l'esperimento `run_live_prediction_experiment` mette in campo 6 robot-token, ne guasta 3 con un `preavviso_intermittente` di 60s, e misura sia se il robot compare su `anomalies` con `type="previsione"` sia la latenza fra l'istante reale di attivazione del guasto e il momento in cui l'anomalia viene ricevuta. Sull'ultimo run: precision=1.00, recall=1.00, F1=1.00 (TP=3, FP=0, FN=0, TN=3) — nessun falso positivo, a differenza della detection di salute discussa sopra. La latenza media `onset` → previsione osservata è stata di 62.2s, coerente con la soglia "al terzo superamento in una finestra di 60s" della logica descritta in §2.6: un preavviso richiede, per costruzione, di accumulare evidenza su più cicli prima di scattare, quindi una latenza dell'ordine della finestra stessa è attesa, non un difetto.

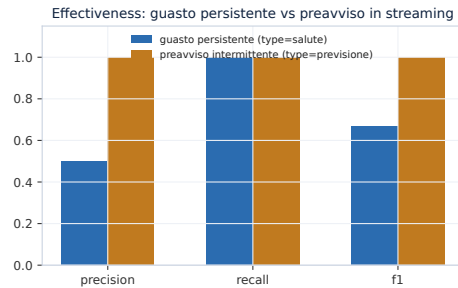


Fig. 4 — Effectiveness a confronto: la detection di salute su guasto persistente (blu, §3.1) soffre di falsi positivi (precision bassa) che il preavviso in streaming (ambra) non mostra in questo run — il recall resta comunque a 1.00 in entrambi i casi.

Deadlock e livelock: verifica di scenario, non una confusion matrix

A differenza della detection di salute, deadlock e livelock **non hanno** un esperimento di precision/recall quantitativo in `eval/`: non esiste un modo economico di generare, in modo ripetuto e controllato, decine di episodi di stallo comportamentale veri e falsi su cui calcolare una statistica (a differenza di un guasto di salute, che si inietta sommando un valore a un canale). La loro correttezza è invece verificata nella suite `test/` (pass/fail, non numeri) con scenari costruiti apposta: test che devono *far scattare* l'anomalia (es. un robot fermo con `task_state="moving"` per oltre 60s consecutivi) e test che *non devono farla scattare* (es. robot in movimento normale, o durante un sorpasso su corridoio a corsia singola). È lo stesso principio di ground truth — uno scenario con uno stato atteso noto a priori — applicato in modo booleano invece che statistico. I tre bug di falsi positivi corretti sul rilevatore di livelock (§2.6) sono stati tutti trovati esattamente in questo modo: un test "non deve scattare" che scattava.

Previsione — come si misura l'errore

Anche qui la verità di riferimento è costruita, non osservata: tre segnali sintetici con **pendenza nota a priori** (es. `motor_temp` che cresce di 0.05°C/s da un valore iniziale noto). Da pendenza e valore iniziale si calcola **analiticamente** l'istante in cui la retta vera incrocerebbe la soglia critica del canale, e da lì il lead time "vero" (`expected_lead_time_s = tempo di incrocio atteso - durata della finestra osservata`). Si dà in pasto lo stesso segnale a `predictive/forecast_failures.py` (§2.8) e si confronta il suo lead time previsto con quello atteso: `errore = lead_time_previsto - lead_time_atteso`. L'MAE (1.6s) è la media del valore assoluto di questo errore sui tre scenari — un numero piccolo perché il segnale è, per costruzione del test, una retta pulita senza rumore: è una misura della correttezza del calcolo analitico d'incrocio, non (solo) della qualità del fit su dati rumorosi reali.

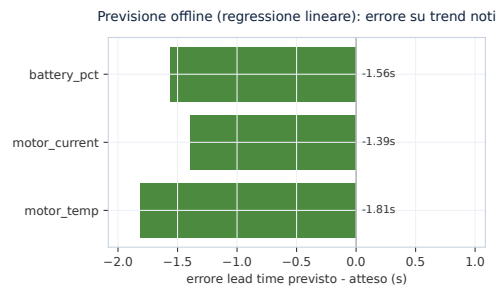


Fig. 5 — Errore fra lead time previsto e lead time atteso sui tre scenari sintetici a pendenza nota: sempre negativo (la previsione è leggermente prudente, cioè anticipa la scadenza reale) e sotto i 2 secondi in valore assoluto su orizzonti di centinaia di secondi.

TAG — come si stabilisce se una risposta è corretta

Per ciascuna delle 22 domande di riferimento (`eval/reference_questions.py`) esiste una query SQL "di verità" scritta a mano. Lo script esegue **entrambe** le query sugli stessi dati storici: quella di riferimento direttamente via Spark SQL, e quella generata dal layer TAG passando dalla domanda in linguaggio naturale. Il confronto fra i due risultati (funzione `question_correct / row_matches`) non richiede un match testuale esatto della query SQL generata (due query sintatticamente diverse possono produrre lo stesso risultato corretto) — confronta invece **le righe restituite**: ogni riga della verità di riferimento deve avere una riga corrispondente nel risultato del TAG, con i valori numerici confrontati con una tolleranza (± 0.5 , per assorbire arrotondamenti) e i valori testuali confrontati per uguaglianza esatta. Una domanda è "corretta" solo se *tutte* le righe attese trovano una corrispondenza — non una corrispondenza parziale.

Limite di scopo dichiarato: questo confronto misura la correttezza dei primi due stadi del layer TAG (query synthesis + query execution, §2.9) — le righe restituite, non la frase in linguaggio naturale del terzo stadio (answer synthesis). Non esiste ancora una metrica automatica che verifichi se la risposta sintetizzata rispecchi fedelmente le righe (un compito di per sé non banale, verificare che un LLM non stia parafrasando in modo fuorviante l'output di un altro LLM) — verificato finora solo a campione, manualmente (§2.9). È uno sviluppo naturale futuro (§4.3), non necessario per dimostrare che lo stadio funziona: il prompt istruisce esplicitamente a non inventare valori assenti dalle righe fornite, e i casi provati a mano (risultato singolo, nessun risultato, gruppo di righe) hanno prodotto risposte fedeli ai dati.

Metrica	Valore	Dettaglio
Detection salute — precision	0.50	8 robot sintetici, 3 con guasto noto (<code>spike_corrente</code>), 5 senza; 3 falsi positivi su 5 robot sani
Detection salute — recall	1.00	0 falsi negativi
Detection salute — F1	0.67	TP=3, FP=3, FN=0, TN=2
Previsione live (streaming) — precision/recall/F1	1.00 / 1.00 / 1.00	6 robot sintetici, 3 con <code>preavviso_intermittente</code> ; TP=3, FP=0, FN=0, TN=3
Previsione live — latenza media onset → previsione	62.2 s	coerente con la finestra di accumulo di 60s del quarto operatore stateful (§2.6)
Previsione offline — errore medio assoluto (MAE)	1.6 s	su lead time previsti di centinaia di secondi (3 scenari sintetici a pendenza nota)
TAG — execution accuracy	20/22 (91%)	22 domande di riferimento in linguaggio naturale, confronto con SQL di verità scritto a mano

Una delle due domande TAG "sbagliate" in questo run è la stessa ambiguità già nota ("qual è la temperatura media dei robot con guasti") — la query di riferimento calcola la media su tutta la telemetria dei robot coinvolti, mentre il modello sceglie, in modo altrettanto legittimo, di calcolarla solo sulle anomalie di temperatura rilevate; è stata lasciata così come esempio concreto dei limiti intrinseci dell'interrogazione in linguaggio naturale, invece di correggere la domanda per farla "tornare" artificialmente. In run diversi il numero di domande corrette è oscillato fra 20 e 21 su 22 — un'ulteriore conferma che l'unico vero limite sistematico è quell'ambiguità dichiarata, non un errore ricorrente sulle stesse domande.

Limite metodologico dichiarato. Le metriche di detection sopra sono misurate sul generatore sintetico (Passo 12), non sulla flotta ROS/Gazebo reale: la ground truth richiede un numero di robot e di ripetizioni controllate che sarebbe troppo lento/costoso da ottenere avviando ripetutamente l'intera simulazione fisica. È un compromesso dichiarato, non nascosto: la pipeline di detection è identica sui due percorsi (stesso schema, stesso codice Spark), e la fedeltà fisica della sorgente ROS/Gazebo è stata comunque verificata qualitativamente e in modo end-to-end (§3.4), ma le cifre di precision/recall riportate qui non provano che la detection funzioni altrettanto bene sul rumore realistico della simulazione fisica quanto su quello, più pulito, del generatore sintetico.

3.2 Efficiency: scalabilità e latenza

Throughput — come si definisce il punto di rottura

Lo sweep (`eval/run_efficiency.py` , funzione `run_throughput_sweep`) lancia il generatore sintetico a carichi crescenti (50, 100, 250, 500, 1000, 2000, 4000, 6000 robot-token, tutti a 10 Hz — target nominale $\text{num_robot} \times \text{hz}$ messaggi/secondo) e, per ciascun livello, legge dal generatore stesso quanti messaggi ha effettivamente pubblicato nell'intervallo (`achieved_rate_msgs_s` , calcolato lato produttore, non stimato lato consumatore). Il rapporto $\text{ratio} = \text{raggiunto} / \text{target}$ è la misura di quanto il sistema regge quel livello di carico; il **punto di rottura** è definito come il primo target per cui ratio scende sotto una soglia fissa dell'80% (`BREAKING_POINT_RATIO = 0.8`) — una soglia arbitraria ma dichiarata, non dedotta a posteriori dai dati.

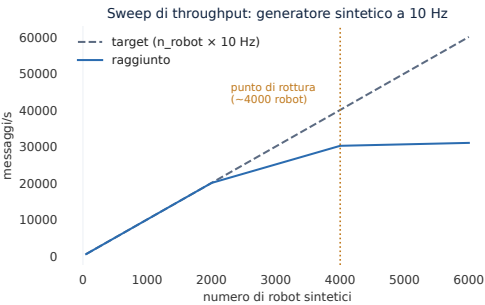


Fig. 6 — Sweep di throughput: la curva raggiunta (blu) segue il target (grigio tratteggiato) fino a ~2000 robot, poi si stacca — il punto di rottura all'80% del target cade a ~4000 robot, dopo il quale il generatore satura a circa 31 000 msg/s indipendentemente da quanti robot in più vengono aggiunti.

Latenza onset → alert — come si misura

Per ciascuna delle 5 prove (`run_latency_trials`): si inietta un guasto `spike_corrente` su un singolo robot-token e si legge, dal topic `injected_faults` , il timestamp reale di attivazione (`start_ts` , scritto dal codice che applica il guasto, non l'istante pianificato). Si attende quindi il primo evento su `anomalies` per quello stesso robot e si registra l'istante in cui lo script di test lo riceve. La latenza riportata è la differenza fra questi due istanti.

Metrica	Valore	Dettaglio
Throughput — scaling pulito fino a	20 000 msg/s	100% del target raggiunto
Throughput — punto di rottura	~40 000 msg/s target	raggiunto 30 968 msg/s, 75% del target (sotto l'80%); a 6000 robot il rapporto scende al 52%
Latenza onset → alert	media 30.3 s (3/5 prove riuscite)	misura volutamente prudente/larga, vedi nota sotto

Il collo di bottiglia individuato per il throughput è il processo Python singolo del generatore sintetico (non Kafka o Spark, che nei test non mostrano saturazione alla stessa scala). Il numero di "30s" di latenza onset → alert **non è la vera latenza minima del sistema** (osservata altrove, in condizioni dedicate, dell'ordine di pochi secondi, coerente col trigger di 2s della query di salute): il meccanismo di misura descritto sopra usa due consumer Kafka interrogati **in sequenza** (prima si attende la conferma del guasto, poi si comincia ad ascoltare gli alert), quindi il numero misura anche il tempo speso dallo script di test stesso prima di mettersi in ascolto, non solo il tempo che il sistema impiega a produrre l'alert — un limite dichiarato del metodo di misura, non delle prestazioni reali (§4.3 propone una misura più rigorosa come sviluppo futuro).

Latenza del loop di auto-riparazione sulla flotta reale (Passo 15/16)

A differenza delle due latenze sopra (misurate sul generatore sintetico), `run_selfhealing_latency_experiment` lavora sulla flotta ROS/Gazebo reale: richiede una simulazione già avviata (verificata via `GET /api/fleet-control/sim/status` prima di procedere — altrimenti l'esperimento viene saltato, non fallito) e misura, su due robot reali distinti presi da `config/experiment.json`, il tempo fra la comparsa dell'anomalia `type="previsione"` (stesso segnale della Fig. 4) e il momento in cui il robot riceve effettivamente il comando di andare al nodo di riparazione (`goal_node == repair_node` osservato sul topic `fleet_state`) — cioè l'unica delle tre latenze misurate end-to-end sull'intero anello percezione → decisione → attuazione, non solo sulla percezione. Sulle due prove eseguite: latenza media 20.2s, entrambe riuscite.

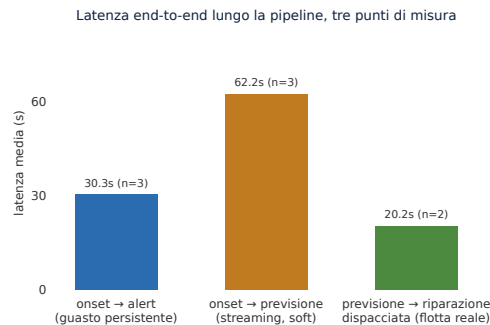


Fig. 7 — Le tre latenze misurate in questo progetto non sono comparabili come "numero unico": misurano tratti diversi della pipeline (percezione grezza, accumulo di evidenza in streaming, intero anello fino all'attuazione fisica) con metodi di misura diversi (§3.2, nota sul metodo prudente della prima barra).

3.3 Valutazione sulla scalabilità

Il sistema ha due assi di scala molto diversi fra loro, con colli di bottiglia diversi — vale la pena distinguerli invece di parlare di "scalabilità" come se fosse un unico numero.

Scala del volume di dati (ingestion/detection): il generatore sintetico

Kafka e Spark, nei test dello sweep (§3.2), non hanno mai mostrato di essere il collo di bottiglia fino a decine di migliaia di messaggi al secondo — il limite osservato (~40 000 msg/s target, 75% raggiunto) è nel **generatore sintetico stesso**: un singolo processo Python (§2.5), limitato dal GIL e dalla schedulazione a coda di priorità di migliaia di robot-token nello stesso processo. La strada di scala orizzontale già prevista nel codice, se servisse spingersi oltre, è lanciare più istanze del generatore in parallelo (prefissi `robot_id` diversi) invece di riscrivere lo script con multiprocessing — non è stato necessario esercitarla per questo progetto. Lato Kafka, la scelta della chiave `robot_id` (§2.4) predispone ad aumentare le partizioni per parallelizzare ulteriormente la lettura, un margine non ancora sfruttato nella configurazione attuale (un solo broker, partizione singola di default).

Il throughput regge, ma la qualità della detection sotto carico no: `run_scalability_experiment`

Il throughput da solo non dice se il sistema resta *corretto* mentre il carico sale: `run_scalability_experiment` misura le due cose insieme, iniettando sempre esattamente 3 guasti reali (`spike_corrente`) su un numero di robot sintetici crescente (100, 1500, 5000, tutti a 3 Hz) e calcolando precision/recall della detection di salute a ciascun livello.

Robot sintetici	Throughput raggiunto	Precision	Recall	TP / FP / FN
100	300 msg/s (100% target)	0.043	1.00	3 / 66 / 0
1500	4500 msg/s (100% target)	0.007	0.67	2 / 272 / 1
5000	14 999 msg/s (100% target)	0.004	1.00	3 / 737 / 0

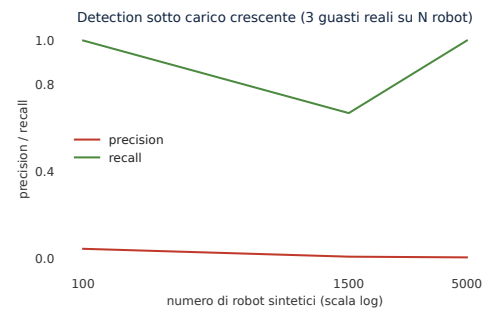


Fig. 8 — Precision e recall della detection di salute mentre il numero di robot sintetici cresce da 100 a 5000, a parità di soli 3 guasti reali iniettati: il recall resta alto (quasi sempre 1.00), la precision crolla verso zero.

Perché la precision crolla, e perché non è (solo) un difetto: il numero di veri positivi resta fisso a 3 per costruzione dell'esperimento (sempre e solo 3 robot guastati, indipendentemente da N), mentre il numero di falsi positivi cresce con N — nei tre livelli il tasso di falso allarme per singolo robot sano è risultato rispettivamente ~68%, ~18%, ~15%, non trascurabile a nessuna scala. La causa è la stessa individuata nella variabilità di run-to-run di §3.1: queste soglie di salute sono **quelle di default** (§2.7), non quelle calibrate su storico da `adaptive_thresholds.py`, perché un run sintetico di poche decine di secondi non ha né il tempo

né lo storico per una calibrazione — il random walk della telemetria sintetica, a 3 Hz per soli 20 secondi, sconfinava abbastanza spesso oltre una soglia fissa da produrre un flusso di falsi positivi che, sommato su migliaia di robot, seppellisce i 3 veri. Il recall resta comunque solido (il segnale vero, uno spike deciso, resta distinguibile dal rumore anche a scala alta): è la precision, non la capacità di individuare i guasti reali, a pagare il prezzo di soglie statiche su un carico sintetico ad alta cardinalità e osservazione breve — un limite del metodo di calibrazione delle soglie in questo scenario di test, non della logica di detection in sé (§4.2).

Scala del numero di robot reali (ROS/Gazebo): un problema diverso, di CPU condivisa

Qui il vincolo non è il throughput dei dati (pochi robot $\times \sim 2\text{Hz}$ è un carico minuscolo per Kafka/Spark) ma la **simulazione fisica**: Gazebo deve calcolare la dinamica, il lidar e la pianificazione locale di ogni robot in tempo reale, un costo che cresce con il numero di robot indipendentemente da quanto sia leggero il resto della pipeline. Il progetto offre due scenari selezionabili (`small` : 4 robot, `large` : 8 robot, §2.11) proprio perché aggiungerne altri non è una questione di codice ma di CPU disponibile sulla macchina host.

Un problema concreto emerso *durante lo sviluppo* illustra bene perché i due assi di scala non sono indipendenti: il valore `spark.cores.max` di `detection.job.py`, alzato a 10 per reggere il carico pesante del generatore sintetico (migliaia di robot-token), si è rivelato eccessivo per la flotta ROS reale — con nessun limite di risorse impostato sui container (§2.4), Spark e Gazebo competono liberamente per la CPU della stessa macchina, e 10 core dati a Spark hanno prodotto micro-batch che invece di 2 secondi ne impiegavano 30-65, osservato nei log (`WARN ProcessingTimeExecutor: Current batch is falling behind`) e confermato misurando l'offset del topic `fleet_state` fermo per decine di secondi alla volta nonostante Gazebo continuasse a muovere i robot normalmente. Ridotto a 4 core, il problema è attenuato ma — onestamente — non risolto del tutto sull'hardware di sviluppo usato per questa verifica (si veda il limite dichiarato in §4.2): un sintomo pratico di quanto la co-localizzazione di un cluster Spark e di un simulatore fisico sulla stessa macchina, senza isolamento di risorse esplicito, sia intrinsecamente più fragile di scalare l'uno o l'altro in isolamento.

In sintesi: il sistema scala bene sull'asse "quanti dati" (Kafka/Spark, dimostrato fino a decine di migliaia di msg/s), assai meno sull'asse "quanti robot fisici simulati contemporaneamente" (limitato dalla CPU della simulazione stessa, aggravato dalla contesa con Spark) — una distinzione che vale la pena avere chiara: aggiungere hardware o ottimizzare il codice risolverebbe il primo problema, ma il secondo resta comunque vincolato dal costo intrinseco di simulare fisica realistica, non solo di elaborare dati.

3.4 Verifica end-to-end sulla flotta reale

Oltre alle metriche quantitative sopra (misurate sul generatore per motivi di scala, §3.1), il comportamento end-to-end è stato verificato con dati realmente prodotti dalla simulazione fisica ROS/Gazebo, non solo letto dal codice:

- Conflitto fisico reale**: due robot comandati verso l'estremità opposta dello stesso corridoio a corsia singola nello stesso momento hanno prodotto uno stallo reale (`task_state="blocked"` osservato su Kafka, non simulato) — prima conferma empirica che i choke point del grafo hanno un effetto fisico reale, non solo un vincolo dichiarato in un file di configurazione.
- Iniezione di un guasto dal vivo** su un robot reale, tramite la dashboard: visibile nella telemetria reale (`motor_current` salito ben oltre il valore nominale) sullo stesso topic `telemetry` usato per tutto il resto della pipeline.
- Anello di self healing**: l'anomalia di salute su soglia fissa, rilevata su dati reali, ha correttamente inviato il robot colpito verso il nodo di riparazione e dispacciato una riserva disponibile sulla missione originale — con selezione corretta fra più riserve disponibili quando un secondo guasto (pre-schedulato, non iniettato manualmente) è scattato in concomitanza sullo scenario a 8 robot.
- Ripristino manuale**: il comando "rimetti in servizio" da dashboard riporta correttamente il robot riparato verso il nodo di riserva.
- Previsione live e reazione differenziata (Passo 15)**: un guasto `preavviso_intermittente` iniettato su un robot reale ha prodotto un'anomalia `type="previsione"` (canale `motor_current`, valore osservato sopra la soglia morbida e sotto quella dura) che ha correttamente attivato la riparazione preventiva con dispaccio riserva; un guasto istantaneo (`spike_corrente`, rampa di 0.5s) sullo stesso robot di riserva ha invece attivato il freeze — il robot si è fermato dov'era (`task_state="idle"` osservato), senza essere inviato al deposito. Il decommissioning successivo ha rimosso il robot da `/api/fleet` in modo persistente (bug reale trovato e corretto: senza un controllo esplicito, il robot ricompariva al messaggio `fleet_state` successivo, dato che il nodo ROS continua a pubblicarlo pur essendo fermo).
- Fix riserve/deposito/previsione live (Passo 16)**: dopo un fermo e riavvio della simulazione reale, una riserva (R4) già dispacciata in un run precedente è stata ridispacciata correttamente in un run successivo (log confermato), verificando il reset esplicito dello stato in memoria del backend. Un run del generatore sintetico dopo il fix del random walk (3000 messaggi) non ha più prodotto alcuna visita ai nodi `DEPOSITO` / `RISERVA1` / `RISERVA2`, presenti invece nello stesso run prima del fix. Una perturbazione `rumore_sensore` su `motor_current` ha prodotto valori visibilmente più dispersi della norma senza generare alcuna riga in `injected_faults`. Un client WebSocket di prova collegato al pannello streaming live ha ricevuto messaggi campionati da tre topic in una finestra di 8 secondi, coerente col throttling configurato.

3.5 Copertura dei requisiti del corso

Requisito (Topic 7)	Dove è coperto
Ingest di stream continui da sensori	Telemetria TurtleBot3 → nodo-ponte ROS → Kafka
Real-time analytics	Detection in Spark Structured Streaming (salute, deadlock, livelock)
Offline analytics su storico → analisi predittiva	Previsione time-series dei guasti (regressione lineare)
Kafka / Spark Streaming	Ingestion e Structured Streaming, presenti end-to-end
Prediction algorithms su time series	Regressione lineare su trend dei canali di salute (ARIMA valutato e poi semplificato)
LLM (open source)	TAG text-to-SQL con Qwen2.5-Coder-32B via router Hugging Face
Esperimenti: effectiveness + efficiency	Suite di valutazione dedicata, con ground truth oggettiva

4. Osservazioni conclusive

4.1 Lezioni tecniche

1. **Soglie fisse e modelli statistici sono complementari, non intercambiabili.** Nei test mirati del Passo 7, ciascuna tecnica ha rilevato casi che l'altra non copriva. La combinazione è più robusta di scegliere una delle due a priori.
2. **Le semantiche di windowing di Structured Streaming sono sottili e vanno verificate empiricamente, non assunte.** Il rilevatore di livelock ha richiesto tre iterazioni (proiezione continua sull'arco, `outputMode("append")` invece di `"update"`, infine uno stato esplicito per chiave via `applyInPandasWithState`) prima di implementare correttamente la definizione di "stallo prolungato" — un'aggregazione a finestra singola, per quanto ben calibrata, non può esprimere una condizione che deve persistere nel tempo attraverso più finestre.
3. **Un modello con un tasso di falsi positivi strutturale diventa consequenziale solo quando viene collegato a un'azione reale.** L'Isolation Forest (`contamination=0.02`) è stato tranquillamente in produzione come segnale passivo per l'intero progetto; il problema è emerso solo dopo averlo (indirettamente) collegato a un'azione automatica su hardware reale, ed è stato risolto a livello architetturale (cambiare il segnale sorgente), non tarando ulteriormente il modello.
4. **La fedeltà dei dati sintetici di training conta quanto la scelta del modello.** Un campionamento indipendente di feature correlate nella realtà (velocità e distanza da ostacoli, per un robot fermo) ha sotto-rappresentato una combinazione tutt'altro che rara, contribuendo ai falsi positivi discussi sopra.
5. **Un modello più semplice, quando il segnale lo giustifica, non è un compromesso.** La regressione lineare ha prodotto risultati praticamente identici ad ARIMA sullo stesso segnale reale (differenza di 0.5s su un lead time di oltre due minuti), con una tecnica enormemente più semplice da spiegare, verificare e mantenere.
6. **NaN è un float Python legale, ma non è JSON valido.** Una metrica che può assumere legittimamente il valore NaN (es. precisione con TP+FP=0) va sempre sanificata esplicitamente prima della serializzazione: Python la accetta senza errori, ma qualunque consumatore JSON standard (incluso il parser nativo dei browser) fallisce silenziosamente su un token `NaN` letterale nel corpo della risposta.

4.2 Limiti dichiarati

- Le metriche ufficiali di detection (§3.1) sono misurate sul generatore sintetico, non sulla flotta ROS/Gazebo reale, per ragioni di scala/tempo sperimentale — limite dichiarato esplicitamente, non nascosto.
- Nell'anello di self healing, la riserva riparte la missione del robot guasto **da capo**, non dal punto esatto di interruzione.
- Non esiste un secondo livello di riserva: se una riserva stessa si guasta durante una missione, non viene sostituita.
- La larghezza dei corridoi fisici (0.8 m) è stata tarata empiricamente, non validata in modo esaustivo su tutte le combinazioni di traiettorie possibili.
- La soglia di conferma del livelock (60s) è una costante fissa, non adattiva al ritmo operativo della flotta.
- La misura di latenza onset → alert riportata (§3.2) è volutamente prudente per un limite del metodo di misura (consumer interrogati in sequenza), non la vera latenza minima del sistema.
- Nessun limite di risorse (CPU/RAM) è impostato sui container Docker (§2.4, §3.3): su una macchina con pochi core disponibili, Spark e Gazebo competono liberamente per la CPU, con un impatto diretto sulla fluidità della dashboard durante l'uso della flotta reale — attenuato tarando `spark.cores.max`, non eliminato del tutto sull'hardware usato per lo sviluppo.
- Un solo broker Kafka a partizione singola per topic (§2.4): l'ordinamento per `robot_id` è garantito e il margine di scalabilità orizzontale resta disponibile in futuro, ma non è ancora sfruttato nella configurazione attuale.
- Il decommissioning di un robot "in avaria" (§2.11, Passo 15) non termina i suoi processi ROS — resta fermo e invisibile in dashboard, non "smontato"; nessun controllo per-robot dei processi esiste oggi in `fleet_control_service.py`.
- La rimozione individuale di un robot dalla flotta è disponibile solo per la flotta reale, non per i robot-token del generatore sintetico.
- Più robot guasti contemporaneamente convergono sullo stesso nodo di deposito (§2.11, Passo 15) — osservato dal vivo, un limite già presente prima di questa estensione (un solo `repair_node`), non risolto qui.
- Il `lead_time_s` della previsione live in streaming (§2.11, Passo 15) è una stima conservativa fissa, non una regressione come quella offline (§2.8) — il segnale a raffiche non si presta a un fit lineare stabile.
- Previsione e guasto persistente condividono lo stesso debounce sul robot colpito: se una previsione ha già azionato la riparazione preventiva, un guasto persistente confermato subito dopo sullo stesso robot non lo ferma una seconda volta (è già in uscita verso il deposito) — scelta deliberata per evitare azioni doppie, non un bug.
- Le perturbazioni (`rumore_sensore`, Passo 16) sul generatore sintetico usano sempre il canale di default (`motor_current`): il form pre-run del generatore non espone la scelta del canale, a differenza del form reattivo della flotta reale.
- Il pannello streaming live (Passo 16) è volutamente campionato (al più un messaggio ogni ~200ms per topic) per non appesantire backend/browser — non rappresenta il volume reale ad alto carico (§3.3).
- Kafka non ha un volume Docker persistente (Passo 16): un riavvio dei container azzerà tutti i topic, mentre lo storico Parquet e i checkpoint (su un volume separato) sopravvivono — un riavvio imprevisto richiede di ripulire manualmente checkpoint/metadata del job di persistenza prima di rilanciarlo.

4.3 Sviluppi futuri

- Tracciare l'avanzamento esatto della missione interrotta, per permettere a una riserva di riprendere da dove il robot guasto si era fermato invece di ripartire da capo.

- Un secondo livello di riserva, per tollerare il guasto di una riserva già dispacciata.
- Calibrazione delle soglie adattive per singolo robot, non solo a livello di flotta, se il volume di storico per robot crescesse a sufficienza.
- Una misura di latenza onset → alert più rigorosa, con consumo parallelo (non sequenziale) dei topic coinvolti.
- Ripetere la misura di precision/recall della detection direttamente sulla flotta ROS/Gazebo reale, per rimuovere il limite metodologico dichiarato in §3.1/§4.2.
- Un token Hugging Face dedicato al progetto, in sostituzione di quello di sviluppo attualmente riusato da un altro progetto dell'autore.
- Una metrica automatica di fedeltà per lo stadio di answer synthesis del layer TAG (§2.9, §3.1): verificare a campione che la risposta in linguaggio naturale rispecchi davvero le righe restituite, non solo che le righe stesse siano corrette.
- Più nodi/bay di riparazione, per evitare che robot guasti contemporaneamente convergano sullo stesso deposito (§2.11, §4.2, Passo 15).
- Un controllo per-robot dei processi ROS in `fleet_control_service.py`, per un decommissioning che fermi davvero il nodo invece di limitarsi a nascondere dalla dashboard.

Relazione redatta passo per passo durante lo sviluppo (vedi [docs/passi/](#) per il dettaglio di ciascuna fase) e verificata empiricamente ad ogni fase.